

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16605-126804

Využitie generatívnej umelej inteligencie v Unity

Bakalárska práca

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16605-126804

Využitie generatívnej umelej inteligencie v Unity

Bakalárska práca

Študijný program:	aplikovaná informatika
Študijný odbor:	informatika
Školiace pracovisko:	Fakulta elektrotechniky a informatiky
Školiteľ:	Ing. Patrik Šebeš



ZADANIE BAKALÁRSKEJ PRÁCE

Študent: **Matej Hrušovský**
ID študenta: 126804
Študijný program: aplikovaná informatika
Študijný odbor: informatika
Vedúci práce: Ing. Patrik Šebeš
Vedúci pracoviska: doc. Ing. Milan Vojvoda, PhD.

Názov práce: **Využitie generatívnej umelej inteligencie v Unity**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

Cieľom bakalárskej práce je preskúmať aktuálne možnosti využitia generatívnej umelej inteligencie (GenAI) za účelom vytvárania dynamického obsahu. Cieľom práce je tiež implementácia jedného alebo viacerých menších prototypov demonštrujúcich konkrétne využitie GenAI v Unity.

Úlohy:

1. Oboznámte sa s aktuálnym stavom problematiky. Vypracujte prehľad súčasných generatívnych modelov s rozdelením podľa modality.
2. Analyzujte spôsoby zadávania vstupov a spracovania výstupov modelov. Zhodnoťte, aký typ dynamického obsahu je možné týmito modelmi vytvárať a identifikujte konkrétne aplikačné scenáre na ich využitie.
3. Analyzujte existujúce aplikácie a spôsob, akým GenAI využívajú.
4. Identifikujte a porovnajte dostupné metódy integrácie modelov do prostredia Unity.
5. Navrhňte a implementujte v prostredí Unity jeden komplexnejší alebo viacero menších prototypov.
6. Otestujte a zhodnoťte implementované riešenie z pohľadu technickej náročnosti a kvality generovaného obsahu. Diskutujte o obmedzeniach a potenciálnych rizikách.

Zoznam odbornej literatúry:

1. BURTON, Brian G. AI for Game Developers: The 2025 Practical Guide. Burtons Media Group, 2025. 226 s. ISBN 978-1-937336-28-8.
2. GUPTA, Shesh Narayan. Generative AI for Game Development: Crafting Narrative Worlds with Machines. New York : Apress, 2026. 293 s. ISBN 979-8-8688-2438-8.
3. LAKSHMANAN, Valliappa; HAPKE, Hannes. Generative AI Design Patterns: Solutions to Common Challenges When Building GenAI Agents and Applications. Sebastopol : O'Reilly Media, 2025. 506 s. ISBN 979-8-341-62266-1.

Podakovanie

Rád by som sa poďakoval svojmu školiteľovi Ing. Patrikovi Šebešovi za odborné vedenie, pripomienky a konzultácie počas tvorby bakalárskej práce.

Abstrakt

Táto bakalárska práca sa zaoberá využitím generatívnej umelej inteligencie pri tvorbe dynamického obsahu v prostredí Unity. Cieľom práce je preskúmať možnosti zapojenia generatívnych modelov do procesu tvorby herného obsahu a navrhnúť prototyp, ktorý demonštruje ich praktické využitie v 2D hre.

Výsledkom práce je prototyp 2D platformovej hry, v ktorom používateľ zadá textovú tému úrovne. Jazykový model následne vytvorí štruktúrovaný plán obsahujúci prompty a herné parametre. Na základe tohto plánu sa pomocou textovo-obrazového modelu generujú vizuálne assety, napríklad pozadie, terénna textúra, hráč, nepriatelia a zberateľné objekty. Unity časť zabezpečuje procedurálne vytvorenie úrovne, spracovanie výstupov, odstránenie pozadia, aplikáciu assetov a prepojenie s hernou logikou.

Práca zároveň hodnotí technickú náročnosť riešenia, úspešnosť jednotlivých častí generovania a kvalitu vytvorených výstupov. Výsledky ukazujú, že generatívna umelá inteligencia môže byť využiteľná pri tvorbe dynamického herného obsahu, avšak jej výstupy si vyžadujú validáciu, postprocessing a náhradné mechanizmy, aby bola zachovaná hrateľnosť a stabilita výsledného prototypu.

Kľúčové slová

generatívna umelá inteligencia, Unity, Stable Diffusion, Ollama, procedurálne generovanie, herné assety, 2D platformer

Abstract

This bachelor thesis focuses on the use of generative artificial intelligence for dynamic content creation in Unity. The goal of the thesis is to explore the possibilities of integrating generative models into the game content creation process and to design a prototype demonstrating their practical use in a 2D game.

The result of the thesis is a prototype of a 2D platformer game in which the user enters a textual theme for a level. A language model then creates a structured plan containing prompts and gameplay parameters. Based on this plan, a text-to-image model generates visual assets such as the background, terrain texture, player character, enemies and collectible objects. The Unity part of the system handles procedural level generation, output processing, background removal, asset application and integration with gameplay logic.

The thesis also evaluates the technical complexity of the solution, the success rate of individual generation steps and the quality of the generated outputs. The results show that generative artificial intelligence can be used for dynamic game content creation, but its outputs require validation, postprocessing and fallback mechanisms in order to preserve the playability and stability of the resulting prototype.

Keywords

generative artificial intelligence, Unity, Stable Diffusion, Ollama, procedural generation, game assets, 2D platformer

Obsah

Úvod	12
1 Teoretické východiská	13
1.1 Prostredie Unity a tvorba herných assetov	13
1.2 Umelá inteligencia v počítačových hrách	14
1.3 Procedurálne generovanie obsahu	14
1.4 Generatívna umelá inteligencia	15
1.5 Rozdelenie generatívnych modelov podľa modality	15
1.6 Veľké jazykové modely	16
1.7 Textovo-obrazové generatívne modely	17
1.8 Komunikácia medzi Unity a externými AI službami	17
2 Prístupy využitia AI v Unity	18
2.1 Rozdelenie podľa času spracovania	18
2.2 Rozdelenie podľa miesta výpočtu	18
2.3 Rozdelenie podľa funkcie v hernom systéme	18
2.4 Rozhodovanie a riadenie správania	19
2.5 Navigácia a pohyb	19
2.6 Učenie a adaptácia	19
2.7 Porovnanie prístupov z pohľadu navrhovaného riešenia	20
2.8 Metódy integrácie generatívnych modelov do Unity	20
3 Generovanie obsahu v hernom vývoji	22
3.1 Prečo generovať obsah	22
3.2 Procedurálne a generatívne prístupy	23
3.3 Typy generovateľného obsahu	23
3.4 Existujúce aplikácie využívajúce GenAI v hernom vývoji	24
3.5 Vstupy a výstupy generatívnych modelov	25
3.6 Požiadavky na použiteľnosť generovaných assetov	26
3.7 Limity a riziká generatívneho prístupu	27
4 Návrh riešenia	28
4.1 Ciele navrhovaného riešenia	28
4.2 Požiadavky na systém	28
4.2.1 Funkčné požiadavky	28
4.2.2 Technické požiadavky	29
4.2.3 Používateľské požiadavky	29

4.3	Analýza možností nasadenia AI backendu	29
4.4	Voľba konkrétnych nástrojov	30
4.4.1	LLM vrstva	30
4.4.2	Obrazová vrstva	31
4.5	Architektúra systému	31
4.5.1	LLM modul	32
4.5.2	Obrazový modul	33
4.5.3	Unity integračný modul	33
4.6	Dátový kontrakt medzi LLM a Unity	33
4.7	Sekvenčné spracovanie pipeline	34
4.8	Riziká návrhu a navrhnuté opatrenia	36
4.9	Zhrnutie návrhu	37
5	Implementácia riešenia	38
5.1	Východiskový stav a implementačný cieľ	38
5.2	Štruktúra Unity projektu	38
5.3	Procedurálne generovanie úrovne	39
5.4	Orchestrácia generovacej pipeline	40
5.5	Integrácia Ollama a generovanie štruktúrovaného plánu	41
5.5.1	Voľba modelu	41
5.5.2	Komunikácia s API a validácia výstupu	41
5.6	Integrácia Stable Diffusion a generovanie obrazových assetov	42
5.7	Voľba checkpointu Stable Diffusion	43
5.8	Postprocessing vygenerovaných assetov	43
5.9	Experiment s generovaním animácií	44
5.10	Aplikácia assetov do scény a herná vrstva	45
5.11	Stabilizácia pipeline a ladenie parametrov	46
5.12	Zhrnutie implementácie	46
6	Testovanie a vyhodnotenie	48
6.1	Testovacie prostredie	48
6.2	Metodika testovania	49
6.3	Testovacia sada	49
6.4	Vyhodnotenie funkčnosti generovacej pipeline	50
6.5	Časová náročnosť generovania	50
6.6	Kvalitatívne hodnotenie generovaných výstupov	51
6.7	Ukážky výsledných scén	53
6.8	Diskusia výsledkov a limity	54

Záver	56
Literatúra	57
Použitie nástrojov umelej inteligencie	58
A Súborová štruktúra projektu	60
B Používateľský manuál	61
B.1 Potrebne nástroje	61
B.2 Príprava Unity projektu	61
B.3 Príprava Ollama	62
B.4 Príprava Stable Diffusion WebUI	62
B.5 Pridanie checkpointu	62
B.6 Príprava rembg	63
B.7 Poradie spúšťania	63
B.8 Spustenie hry	64
B.9 Najčastejšie problémy	64

Zoznam obrázkov a tabuliek

Obrázok 1	Základná štruktúra Unity projektu [4].	14
Obrázok 2	Schéma textovo-obrazového generovania pomocou difúzneho modelu [10].	17
Obrázok 3	Rozdelenie nákladov na vývoj AAA hry podľa oblastí [6]. . .	22
Obrázok 4	Zjednodušená architektúra systému a vzťahy medzi hlavnými modulmi.	32
Obrázok 5	Sekvenčný diagram hlavného priebehu generovania obsahu. .	35
Obrázok 6	Výsledok procedurálneho generovania platformového rozloženia.	40
Obrázok 7	Ukážka nekonzistentného spritesheetu.	44
Obrázok 8	Porovnanie pôvodnej referenčnej postavy a výstupu získaného pomocou režimu <code>img2img</code>	45
Obrázok 9	Ukážky generovaných výstupov použitých pri kvalitatívnom hodnotení	52
Obrázok 10	Výsledná scéna pre tému Cave	53
Obrázok 11	Výsledná scéna pre tému Volcano	53
Obrázok 12	Výsledná scéna pre tému Forest	54
Tabuľka 1	Prehľad generatívnych modelov podľa modality	16
Tabuľka 2	Rozdelenie AI prístupov podľa času spracovania a miesta výpočtu	20
Tabuľka 3	Porovnanie metód integrácie generatívnych modelov do Unity	21
Tabuľka 4	Porovnanie existujúcich GenAI nástrojov z pohľadu herného vývoja	25
Tabuľka 5	Vstupy a výstupy generatívnych modelov v hernom vývoji .	26
Tabuľka 6	Porovnanie možností nasadenia AI backendu	30
Tabuľka 7	Porovnanie zvažovaných nástrojov pre jednotlivé vrstvy systému	31
Tabuľka 8	Identifikované riziká návrhu a navrhnuté opatrenia	36
Tabuľka 9	Zodpovednosti hlavných tried implementovaného prototypu .	39
Tabuľka 10	Porovnanie testovaných checkpointov Stable Diffusion	43
Tabuľka 11	Zvolená konfigurácia parametrov generovania	46
Tabuľka 12	Testovacie prostredie	48
Tabuľka 13	Testovacia sada tém	49
Tabuľka 14	Úspešnosť jednotlivých častí generovacej pipeline	50
Tabuľka 15	Časová náročnosť jednotlivých krokov generovania	50

Tabuľka 16	Bodové hodnotenie kvality generovaných výstupov	51
------------	---	----

Zoznam značiek a skratiek

AI - umelá inteligencia, z angl. *Artificial intelligence*

GenAI - generatívna umelá inteligencia, z angl. *Generative artificial intelligence*

LLM - veľký jazykový model, z angl. *Large Language Model*

API - aplikačné programové rozhranie, z angl. *Application Programming Interface*

PG - procedurálne generovanie, z angl. *Procedural generation*

PCG - procedurálne generovanie obsahu, z angl. *Procedural content generation*

NPC - nehrateľná postava, z angl. *Non-player character*

FSM - konečné stavové automaty, z angl. *Finite State Machine*

BT - behaviorálne stromy, z angl. *Behavior Trees*

ML - strojové učenie, z angl. *Machine learning*

HUD - herné používateľské rozhranie, z angl. *Head-up display*

HTTP - protokol na komunikáciu medzi aplikáciami, z angl. *Hypertext Transfer Protocol*

JSON - textový formát na výmenu dát, z angl. *JavaScript Object Notation*

UI - používateľské rozhranie, z angl. *User Interface*

REST - architektonický štýl webových rozhraní, z angl. *Representational State Transfer*

Úvod

Generatívna umelá inteligencia sa v posledných rokoch stala významným nástrojom pri tvorbe textového, obrazového, zvukového aj multimodálneho obsahu. Jej využitie sa postupne rozširuje aj do oblasti vývoja počítačových hier, kde môže podporovať tvorbu dialógov, príbehových prvkov, vizuálnych návrhov, textúr, sprite assetov alebo celých prototypov herných scén. V porovnaní s tradičným ručným vytváraním obsahu ponúka generatívna AI možnosť rýchlejšie pripraviť väčšie množstvo variabilných výstupov. Zároveň však prináša problémy spojené s nedeterministickosťou, technickou použiteľnosťou a konzistentnosťou generovaného obsahu.

Prostredie Unity predstavuje vhodnú platformu na uplatnenie takéhoto prístupu, pretože umožňuje pracovať s 2D aj 3D scénami, hernými objektmi, assetmi, fyzikou, používateľským rozhraním a externými službami. V prípade GenAI je však potrebné riešiť nielen samotné volanie modelov, ale aj spracovanie ich výstupov. Obrázok vygenerovaný obrazovým modelom nemusí byť priamo použiteľný ako herný asset a textový výstup jazykového modelu nemusí mať vždy očakávanú štruktúru. Preto je potrebné doplniť validačné mechanizmy, postprocessing a náhradné riešenia.

Cieľom tejto bakalárskej práce je preskúmať možnosti využitia GenAI pri tvorbe dynamického obsahu v Unity a implementovať prototyp demonštrujúci konkrétne použitie tohto prístupu. Práca sa zameriava na prepojenie textového vstupu používateľa, jazykového modelu, obrazového modelu a procedurálne generovanej 2D úrovne. Používateľ zadá tému úrovne, jazykový model z nej vytvorí štruktúrovaný plán a obrazový model následne vygeneruje vizuálne assety, ktoré sa spracujú a aplikujú do hernej scény.

Prvá časť práce sa venuje teoretickým východiskám, najmä prostrediu Unity, herným assetom, procedurálnemu generovaniu, generatívnej umelej inteligencii a rozdeleniu generatívnych modelov podľa modality. Následne sú opísané možnosti využitia umelej inteligencie v Unity, metódy integrácie modelov a existujúce nástroje využívajúce GenAI v hernom vývoji. Praktická časť práce opisuje návrh a implementáciu prototypu, integráciu jazykového modelu cez Ollama, generovanie obrazových výstupov pomocou Stable Diffusion WebUI, postprocessing assetov a ich aplikáciu do Unity scény. Záverečná časť sa venuje testovaniu, časovej náročnosti, úspešnosti generovania, kvalite výstupov a obmedzeniam navrhnutého riešenia.

1 Teoretické východiská

Táto kapitola uvádza teoretické pojmy a technológie potrebné na lepšie pochopenie návrhu a implementácie riešenia. Práca sa zaoberá využitím umelej inteligencie v prostredí Unity, konkrétne automatizovanou tvorbou herných assetov pomocou lokálne prevádzkovaných generatívnych modelov. Z tohto dôvodu sa kapitola nezameriava iba na všeobecné definície umelej inteligencie, ale najmä na tie oblasti, ktoré priamo súvisia s navrhovanou pipeline: herné assety, procedurálne generovanie, veľké jazykové modely, textovo-obrazové modely a komunikáciu medzi Unity a externými AI službami.

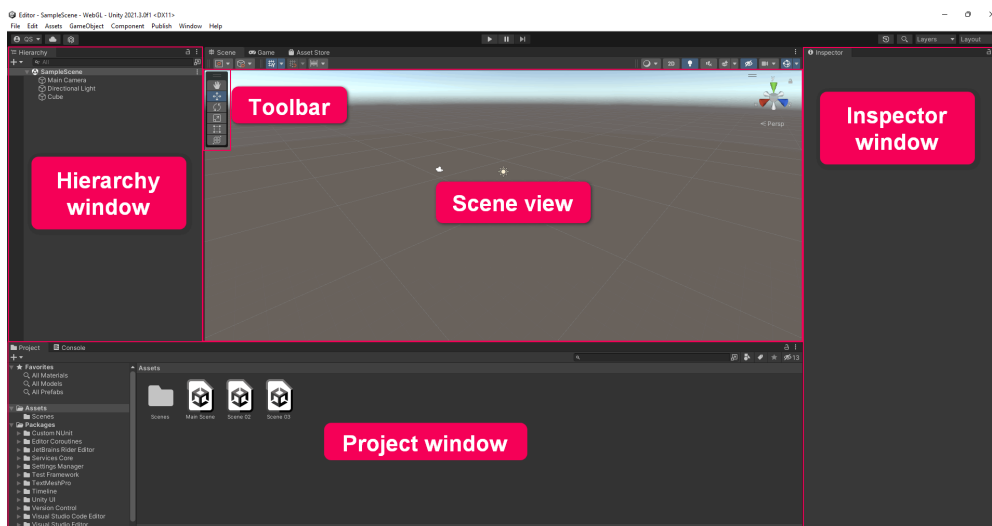
1.1 Prostredie Unity a tvorba herných assetov

Unity je vývojové prostredie a herný engine určený na tvorbu 2D a 3D aplikácií pre rôzne cieľové platformy [1]. Z pohľadu tejto práce je dôležité najmä to, že Unity organizuje herný obsah do scén, objektov a assetov. Scéna predstavuje konkrétnu časť aplikácie alebo hernej úrovne, v ktorej sú umiestnené objekty, komponenty, používateľské rozhranie a herná logika. Základnou entitou v scéne je objekt typu *GameObject*, ktorý slúži ako kontajner pre komponenty určujúce jeho správanie, vzhľad alebo fyzikálne vlastnosti [2].

Asset v Unity označuje akýkoľvek prostriedok používaný v projekte, napríklad textúra, sprite, 3D model, zvuk, materiál alebo dátový súbor [3]. V kontexte tejto práce sú podstatné najmä vizuálne assety.

Opakovateľné herné objekty je možné ukladať ako *prefaby*. Prefab predstavuje šablónu objektu spolu s jeho komponentmi a nastaveniami, z ktorej možno vytvárať ďalšie inštancie v scéne [2]. V prípade hier s procedurálne alebo dynamicky vytváraným obsahom sú prefaby dôležité najmä pri generovaní nepriateľov, zberateľných predmetov alebo iných opakovane používaných objektov.

Z hľadiska 2D hier je dôležitý aj pojem *sprite*. Sprite predstavuje dvojrozmerný grafický objekt, ktorý sa používa napríklad pre postavy, nepriateľov, predmety alebo prvky používateľského rozhrania. Pri generovaní sprite assetov pomocou AI nestačí získať esteticky vhodný obrázok. Výstup musí mať vhodnú mierku, čitateľný tvar, oddelenie od pozadia a musí byť technicky použiteľný v scéne s kolíziami a hernou logikou.



Obr. 1: Základná štruktúra Unity projektu [4].

1.2 Umelá inteligencia v počítačových hrách

Umelá inteligencia predstavuje súbor metód a technológií, ktoré umožňujú počítačovým systémom riešiť úlohy vyžadujúce určitú mieru učenia, rozhodovania alebo adaptácie [5]. V počítačových hrách sa pojem umelá inteligencia používa širšie ako v akademickom výskume. Často označuje aj pravidlové systémy, rozhodovacie stromy, navigačné algoritmy alebo skriptované správanie, ktoré nemusia využívať strojové učenie, ale vytvárajú dojem inteligentného správania.

Tradičné využitie umelej inteligencie v hrách zahŕňa najmä riadenie nehráčskych postáv, navigáciu, výber akcií, adaptáciu obtiažnosti a simuláciu správania herného sveta. Medzi bežné techniky patria konečné stavové automaty, behaviorálne stromy, systémy užitočnosti, pathfinding a rôzne formy plánovania [6]. Tieto prístupy sú často deterministické, dobre kontrolovateľné a vhodné pre systémy, ktoré musia reagovať v reálnom čase.

V posledných rokoch sa však využitie umelej inteligencie v hrách rozšírilo aj na tvorbu obsahu. Generatívne modely dokážu podporovať tvorbu textov, dialógov, vizuálnych návrhov, textúr, sprite assetov alebo herných scenárov. V takom prípade AI neslúži iba na riadenie postáv počas hry, ale stáva sa nástrojom na automatizovanú alebo asistovanú tvorbu herných prvkov.

1.3 Procedurálne generovanie obsahu

Procedurálne generovanie obsahu je prístup, pri ktorom sa herný obsah vytvára pomocou algoritmov namiesto ručnej tvorby každého prvku [7]. Takýmto spôsobom možno

generovať napríklad úrovne, mapy, rozloženie miestností, terén, predmety, nepriateľov alebo iné herné štruktúry.

Výhodou procedurálneho generovania je možnosť vytvárať väčšie množstvo variabilného obsahu s menším manuálnym úsilím. V hrách sa tento prístup používa napríklad na tvorbu rozsiahlych svetov, náhodne generovaných dungeonov alebo opakovateľne hrateľných úrovní. Procedurálne generovanie môže byť deterministické, ak používa pevne definovaný algoritmus a seed hodnotu, alebo adaptívne, ak zohľadňuje stav hry, obtiažnosť alebo správanie hráča.

1.4 Generatívna umelá inteligencia

Generatívna umelá inteligencia je oblasť umelej inteligencie zameraná na tvorbu nového obsahu, napríklad textu, obrázkov, zvuku, videa alebo kódu [8]. Na rozdiel od systémov, ktoré iba klasifikujú alebo analyzujú existujúce dáta, generatívne modely vytvárajú nové výstupy na základe vstupných požiadaviek, príkladov alebo naučených vzorov.

V hernom vývoji môže GenAI podporovať rôzne časti tvorivého procesu. Jazykové modely možno použiť na generovanie dialógov, opisov predmetov, príbehových udalostí alebo štruktúrovaných dát. Textovo-obrazové modely možno využiť pri tvorbe koncept artu, textúr, ikon alebo 2D sprite assetov. V prípade prototypovania hier môže GenAI urýchliť iteráciu medzi návrhom a vizuálnym výsledkom.

Použitie GenAI však prináša aj obmedzenia. Výstupy modelov nemusia byť deterministické, môžu obsahovať artefakty, nemusia dodržiavať požadovanú štruktúru alebo môžu byť nevhodné na priame použitie v hernom engine. Preto je potrebné doplniť generovanie o validáciu, postprocessing, opakované pokusy a fallback mechanizmy.

1.5 Rozdelenie generatívnych modelov podľa modality

Generatívne modely možno rozdeliť podľa typu vstupu a výstupu, s ktorým pracujú. Pre potreby herného vývoja sú dôležité najmä textové, obrazové, zvukové, video, 3D a multimodálne modely. Jednotlivé modality sa líšia typom generovaného obsahu aj spôsobom, akým môžu byť využité pri tvorbe hier.

Tabuľka 1: Prehľad generatívnych modelov podľa modality

Modalita	Typický výstup	Možné využitie v hrách	Príklady modelov alebo nástrojov
Text	dialógy, opisy, pravidlá, JSON dáta	príbeh, dialógy, plánovanie úrovne	GPT, Claude, Gemini, Llama, Mistral
Obraz	obrázky, textúry, ikony, sprity	vizuálne assety, pozadia, koncept art	Stable Diffusion, DALL-E, Midjourney, Adobe Firefly
Zvuk	hudba, efekty, reč	zvukové efekty, dabing, atmosféra	MusicGen, AudioLDM, ElevenLabs, Suno
Video	krátke animácie alebo scény	prototypovanie cutscén, animované ukážky	Sora, Runway, Pika, Kling
3D	modely, tvary, objekty	návrh objektov, prostredí alebo rekvizít	Shap-E, DreamFusion, Meshy, TripoSR
Multimodálne modely	kombinácia textu, obrazu a ďalších dát	komplexnejšie asistované návrhové nástroje	GPT-4o, Gemini, Claude, LLaVA

1.6 Veľké jazykové modely

Veľké jazykové modely sú modely AI trénované najmä na veľkých množstvách textových dát. Ich základnou úlohou je spracovanie a generovanie textu na základe vstupného kontextu. Moderné jazykové modely sú často založené na architektúre transformer, ktorá umožňuje efektívne spracovanie sekvencií a modelovanie vzťahov medzi časťami vstupu.

Pôvodne boli veľké jazykové modely využívané najmä na úlohy typu text-to-text, napríklad odpovedanie na otázky, sumarizáciu, preklad, tvorbu dialógov alebo generovanie štruktúrovaného textu. Súčasné modely sa však čoraz častejšie rozširujú aj o multimodálne schopnosti. Vďaka tomu môžu pracovať nielen s textom, ale aj s obrazovým alebo iným vstupom. V praxi sa tak používajú napríklad pri úlohách typu image-to-text, kde model opisuje obsah obrázka, alebo ako súčasť systémov podporujúcich text-to-image a image-to-image generovanie.

V hernom vývoji možno LLM využiť na generovanie dialógov, opisov predmetov, návrhov misií, pravidiel, naratívnych prvkov alebo štruktúrovaných dát. Dôležité je najmä to, že jazykový model nemusí vytvárať iba prirodzený jazyk, ale môže generovať aj formálne výstupy, napríklad JSON objekty. Takéto výstupy sú vhodné v prípadoch,

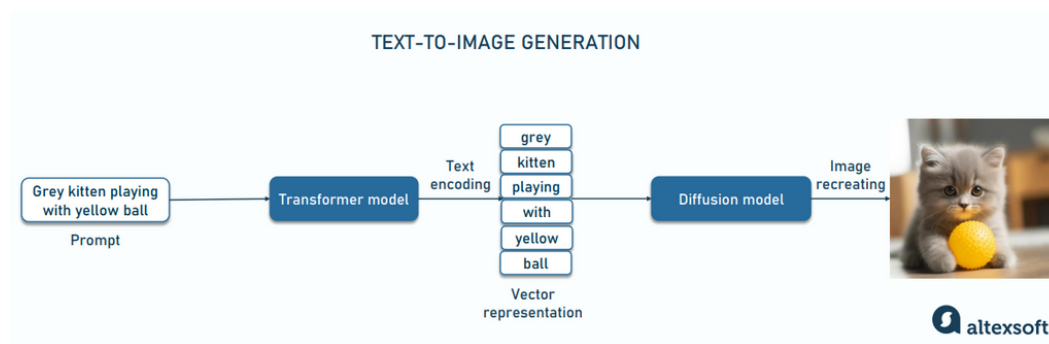
keď má jazykový model slúžiť ako plánovacia alebo koordinačná vrstva pre ďalšie časti systému.

1.7 Textovo-obrazové generatívne modely

Textovo-obrazové modely umožňujú vytvárať obrazový výstup na základe textového opisu. Používateľ alebo nadradený systém zadáva prompt, ktorý opisuje požadovaný obsah, štýl, kompozíciu alebo technické vlastnosti obrázka. Často sa používa aj negatívny prompt, ktorý špecifikuje nežiaduce vlastnosti výstupu, napríklad nesprávnu perspektívu, text v obraze, rámovanie, deformácie alebo iné artefakty.

Medzi významné prístupy v tejto oblasti patria difúzne modely. Difúzne modely generujú obraz postupným odstraňovaním šumu a rekonštrukciou výsledného obrazu na základe naučených pravdepodobnostných vzťahov. Latentné difúzne modely, medzi ktoré patrí aj Stable Diffusion, pracujú v komprimovanom latentnom priestore, čo umožňuje efektívnejšie generovanie obrázkov pri zachovaní vysokej vizuálnej kvality [9].

V kontexte herného vývoja možno textovo-obrazové modely využiť na tvorbu koncept artu, pozadí, textúr, ikon alebo sprite assetov. Pri použití v hernom engine však nestačí hodnotiť iba estetickú kvalitu obrázka. Výstup musí byť technicky použiteľný ako herný asset.



Obr. 2: Schéma textovo-obrazového generovania pomocou difúzneho modelu [10].

1.8 Komunikácia medzi Unity a externými AI službami

Pri integrácii umelej inteligencie do Unity projektu je dôležitá komunikácia medzi hernou aplikáciou a výpočtovým backendom. Táto komunikácia sa často realizuje prostredníctvom aplikačného programového rozhrania API, ktoré definuje spôsob odosielania požiadaviek a prijímania odpovedí medzi softvérovými komponentmi [11].

2 Prístupy využitia AI v Unity

Využitie AI v prostredí Unity možno rozdeliť podľa času spracovania, miesta výpočtu a funkcie, ktorú AI systém v aplikácii plní. Čas spracovania určuje, či sa AI používa počas behu aplikácie alebo ešte pred jej spustením. Miesto výpočtu ovplyvňuje latenciu, hardvérové nároky a závislosť od externých služieb. Funkčné hľadisko rozlišuje, či AI slúži na riadenie správania, navigáciu, učenie alebo generovanie obsahu.

2.1 Rozdelenie podľa času spracovania

Podľa času spracovania možno AI systémy rozdeliť na dynamické a statické. Dynamické systémy pracujú počas behu aplikácie a priamo ovplyvňujú aktuálny stav hry. Patria sem napríklad rozhodovacie systémy NPC, navigácia, dynamická zmena obtiažnosti alebo generovanie obsahu počas hrania. Pri takomto použití je dôležitá najmä nízka latencia a predvídateľný vplyv na výkon.

Statické použitie zahŕňa procesy vykonávané pred spustením aplikácie, napríklad prípravu assetov, tréning modelov alebo offline generovanie obsahu. Výhodou je, že výpočtovo náročné úlohy nezatažujú aplikáciu počas hrania.

2.2 Rozdelenie podľa miesta výpočtu

Podľa miesta výpočtu možno AI riešenia rozdeliť na lokálne, vzdialené a hybridné. Lokálne riešenia bežia na zariadení používateľa alebo na rovnakom počítači ako aplikácia. Poskytujú väčšiu kontrolu nad modelmi, nižšiu závislosť od internetu a jednoduchšiu reprodukovateľnosť, no vyžadujú výkonnejší hardvér.

Vzdialené riešenia využívajú cloudové služby alebo servery tretích strán. Výhodou je jednoduchšie nasadenie výpočtovo náročných modelov, nevýhodou závislosť od internetu, vyššia latencia, možné náklady a menšia kontrola nad verziami modelov.

Hybridný prístup kombinuje lokálne a vzdialené výpočty. Môže byť vhodný pre produkčné riešenia, no zároveň zvyšuje zložitosť návrhu a implementácie.

2.3 Rozdelenie podľa funkcie v hernom systéme

Z funkčného hľadiska možno AI v hrách rozdeliť na systémy pre rozhodovanie, navigáciu, učenie a generovanie obsahu. Rozhodovacie systémy určujú správanie nehráčskych postáv, navigačné systémy riešia pohyb v prostredí, systémy učenia umožňujú adaptáciu správania a generatívne systémy sa zameriavajú na tvorbu nového obsahu.

2.4 Rozhodovanie a riadenie správania

Rozhodovanie a riadenie správania patria medzi tradičné oblasti využitia umelej inteligencie v hrách. Cieľom je, aby nehráčske postavy reagovali na stav prostredia, hráča alebo vlastné vnútorné premenné. Medzi bežné prístupy patria konečné stavové automaty, behaviorálne stromy a systémy užitočnosti.

Konečný stavový automat, označovaný ako FSM, modeluje správanie pomocou stavov a prechodov medzi nimi. NPC sa môže nachádzať napríklad v stave hliadkovania, prenasledovania alebo útoku. Výhodou FSM je jednoduchosť, nevýhodou horšia škálovateľnosť pri komplexnejšom správaní [6].

Behaviorálne stromy predstavujú hierarchický spôsob modelovania správania. Umožňujú lepšiu modularitu a znovupoužiteľnosť častí správania než jednoduché stavové automaty. V Unity ich možno implementovať ručne alebo pomocou špecializovaných nástrojov [6].

Systémy užitočnosti vyberajú akcie na základe skóre vypočítaného z aktuálneho stavu hry, napríklad zdravia, vzdialenosti od hráča alebo dostupnosti zdrojov. Tento prístup umožňuje flexibilnejšie rozhodovanie než pevne definované pravidlá [6].

2.5 Navigácia a pohyb

Navigácia rieši pohyb agentov v hernom prostredí. Úlohou navigačného systému je nájsť vhodnú trasu k cieľu tak, aby sa agent vyhol prekážkam a rešpektoval geometriu scény. Medzi známe algoritmy patrí A^* , ktorý vyhľadáva cestu v grafe alebo mriežke pomocou heuristickej funkcie [12]. V Unity sa pre 3D prostredia často používa NavMesh, zatiaľ čo pri 2D alebo tilemapových hrách je bežný mriežkový prístup.

2.6 Učenie a adaptácia

Systémy učenia umožňujú agentom zlepšovať správanie na základe skúseností. V prostredí Unity je významným nástrojom Unity ML-Agents Toolkit, ktorý umožňuje trénovať agentov v simulovaných scénach [13]. Agent pozoruje stav prostredia, vykonáva akcie a dostáva odmeny alebo tresty podľa úspešnosti správania.

ML-Agents podporuje napríklad posilňovacie učenie a imitačné učenie. Tieto prístupy sú vhodné najmä v situáciách, kde je ťažké ručne navrhnuť optimálne pravidlá správania.

2.7 Porovnanie prístupov z pohľadu navrhovaného riešenia

Nasledujúca tabuľka sumarizuje základné rozdelenie AI prístupov podľa času spracovania a miesta výpočtu. Tabuľka zároveň ukazuje, že GenAI systémy nemusia byť výhradne vzdialené alebo statické. V navrhovanom riešení sa generatívne modely prevádzkujú lokálne a ich výstupy sú pripravované ako súčasť generovania úrovne.

Tabuľka 2: Rozdelenie AI prístupov podľa času spracovania a miesta výpočtu

	Lokálne výpočty	Vzdialené výpočty
Počas behu aplikácie	FSM, behaviorálne stromy, NavMesh, lokálne LLM, lokálne generovanie assetov pred načítaním úrovne	Cloudové LLM dialógy, vzdialené generovanie obrázkov, online adaptívne systémy
Pred spustením aplikácie	Tréning ML modelov, offline procedurálne generovanie obsahu, príprava assetov	Cloudové generovanie assetov, vzdialený tréning modelov, automatizované testovanie v cloude

2.8 Metódy integrácie generatívnych modelov do Unity

Generatívne modely možno do Unity integrovať viacerými spôsobmi. Výber integračnej metódy závisí od toho, či má model bežať lokálne alebo vzdialene, či je potrebné generovanie počas behu aplikácie a akú mieru kontroly nad modelom vývojár potrebuje.

Tabuľka 3: Porovnanie metód integrácie generatívnych modelov do Unity

Metóda integrácie	Výhody	Nevýhody
Cloudové API	jednoduché nasadenie, nízke lokálne hardvérové nároky, dostupnosť kvalitných modelov	závislosť od internetu, náklady, správa API kľúčov, menšia kontrola nad verziou modelu
Lokálny HTTP server	dobrá oddelenosť Unity a AI vrstvy, možnosť lokálneho spustenia, jednoduchá komunikácia cez REST API	vyššia konfigurácia prostredia, nutnosť spúšťať samostatný backend
Unity plugin alebo balík	pohodlná integrácia priamo v editore, jednoduchšie používanie pre vývojára	závislosť od konkrétneho nástroja, menšia flexibilita pri výmene backendu
Priame volanie knižnice alebo procesu	vysoká kontrola nad spustením modelu a parametrami	vyššia implementačná zložitosť, riziko blokovania aplikácie
Offline predgenerovanie assetov	žiadna záťaž počas hrania, jednoduché testovanie výstupov	obsah nie je dynamický počas behu aplikácie, nižšia interaktivita

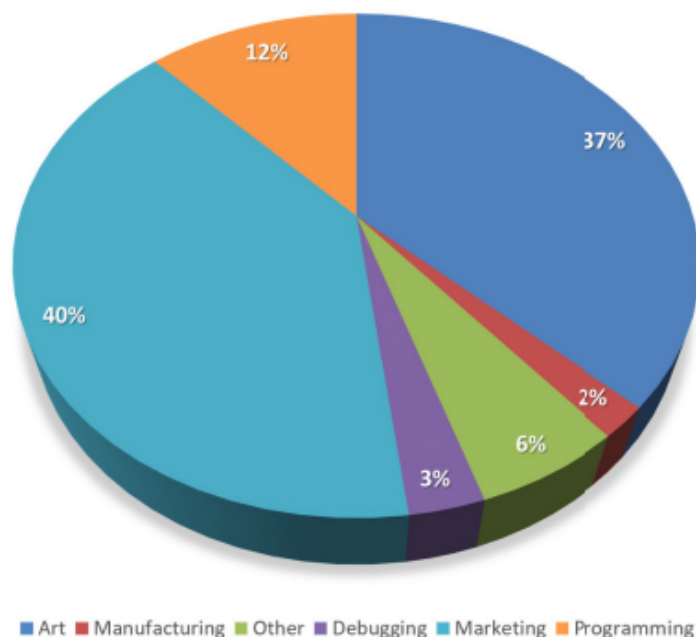
3 Generovanie obsahu v hernom vývoji

Generovanie obsahu predstavuje oblasť, v ktorej sa algoritmické a AI prístupy využívajú na tvorbu herných prvkov. Môže ísť o úrovne, mapy, predmety, postavy, textúry, zvuky, dialógy alebo naratívne štruktúry.

3.1 Prečo generovať obsah

Jedným z hlavných dôvodov využívania generovania obsahu v hrách je znižovanie nákladov a času potrebného na vývoj. Tvorba rozsiahlych herných svetov, úrovní, grafických prvkov alebo iného obsahu si vyžaduje veľa práce dizajnérov a umelcov, čo je pri veľkých hrách finančne aj časovo náročné. Procedurálne generovanie obsahu umožňuje, aby časť tejto práce prebrali algoritmy, vďaka čomu môžu hry vznikať rýchlejšie a lacnejšie. Zároveň to pomáha aj menším tímom alebo jednotlivcom vytvárať obsahovo bohatšie hry bez potreby veľkého rozpočtu [7].

Generovanie obsahu však nemá slúžiť iba ako náhrada ľudskej práce, ale aj ako nástroj na podporu kreativity. Algoritmy dokážu priniesť nové kombinácie, riešenia alebo nápady, ktoré by autor nemusel sám navrhnuť. Okrem toho môžu umožniť vznik hier s priebežne vytváraným obsahom, ktoré sa prispôbujú hráčovi podľa jeho správania alebo schopností. Procedurálne generovanie tak rozširuje možnosti herného dizajnu a zároveň pomáha lepšie pochopiť samotný proces tvorby a kreativity [7].



Obr. 3: Rozdelenie nákladov na vývoj AAA hry podľa oblastí [6].

3.2 Procedurálne a generatívne prístupy

Pri tvorbe herného obsahu možno rozlišovať medzi procedurálnym a generatívnym prístupom. Procedurálne generovanie je založené na explicitne navrhnutých algoritmoch a pravidlách. Vývojár určuje, ako sa má obsah vytvárať, aké obmedzenia musí spĺňať a aké parametre ovplyvňujú výsledok. Tento prístup je dobre kontrolovateľný a vhodný napríklad na tvorbu rozloženia úrovni, miestností, plošín alebo spawn bodov [7].

GenAI vytvára obsah na základe naučených vzorov z tréningových dát. Vývojár neurčuje presný algoritmus tvorby každého prvku, ale zadáva požiadavku vo forme promptu, parametrov alebo ukážok. Výhodou je vysoká variabilita a schopnosť vytvárať vizuálne alebo textové výstupy, ktoré by bolo náročné ručne špecifikovať pravidlami. Nevýhodou je nižšia predvídateľnosť a potreba validácie výsledkov.

3.3 Typy generovateľného obsahu

V hernom vývoji možno generovať rôzne typy obsahu. Textový obsah zahŕňa dialógy, opisy predmetov, úlohy, sprievodné texty alebo tutoriály. Jazykové modely umožňujú tieto texty prispôbovať téme hry, stavu hráča alebo aktuálnej situácii. V Unity môžu byť takéto výstupy ukladané do dátových štruktúr, zobrazované v používateľskom rozhraní alebo použité ako vstup pre ďalšie systémy [7].

Naratívne prvky zahŕňajú príbehové udalosti, vetvenie deja, ciele misií alebo tematické kontexty úrovni. Ich generovanie môže zvýšiť variabilitu hry a umožniť vytváranie obsahu prispôbeného používateľskému vstupu [7].

Vizuálny obsah zahŕňa koncept art, pozadia, 2D sprity, textúry, ikony a používateľské rozhranie. Textovo-obrazové modely dokážu vytvárať návrhy vo viacerých štýloch a rýchlo rozširovať množstvo dostupných vizuálnych variantov. V Unity je možné výsledky použiť ako textúry, sprity alebo podklady pre ďalšie spracovanie [7].

Generovať možno aj herné úrovne a mapy. Pri tradičnom procedurálnom prístupe sa používajú algoritmy založené na pravidlách, gramatikách, mriežkach alebo seed hodnotách. Pri GenAI prístupoch môže model navrhovať tému, rozloženie, typy nepriateľov alebo parametre obtiažnosti [7].

Ďalšou kategóriou sú testovacie a syntetické dáta. AI môže pomáhať pri tvorbe testovacích scenárov alebo simulovaných vstupov. V hernom vývoji môžu takéto dáta slúžiť na overovanie správania agentov, testovanie úrovni alebo ladenie parametrov [7].

3.4 Existujúce aplikácie využívajúce GenAI v hernom vývoji

V hernom vývoji sa GenAI využíva v rôznych typoch nástrojov. Niektoré sú integrované priamo do vývojového prostredia, iné sa zameriavajú na tvorbu vizuálnych assetov, prototypovanie hier alebo lokálne spúšťanie generatívnych modelov. Nasledujúce príklady ukazujú najčastejšie spôsoby praktického využitia GenAI pri tvorbe herného obsahu.

Unity AI predstavuje súbor nástrojov integrovaných do prostredia Unity. Ich cieľom je podporiť vývojára počas práce na projekte, napríklad pri tvorbe, úprave alebo vyhľadávaní assetov a pri zrýchlení vybraných častí vývojového procesu [14].

Scenario je nástroj zameraný najmä na generovanie vizuálnych assetov pre hry. Umožňuje vytvárať postavy, objekty, prostredia, textúry a ďalšie vizuálne výstupy. Využíva sa najmä tam, kde je potrebné rýchlo pripraviť viacero vizuálnych variantov v jednotnom štýle [15].

Rosebud AI sa zameriava na prototypovanie hier pomocou textového zadania. Používateľ opisuje požadovanú hru alebo scénu prirodzeným jazykom a systém z tohto zadania pomáha vytvoriť herný prototyp alebo jeho časti [16].

Stable Diffusion WebUI umožňuje generovať obrázky na základe textových promptov a negatívnych promptov. Používa sa najmä na tvorbu koncept artu, textúr, ikon, pozadí alebo sprite assetov. Výhodou je možnosť lokálneho spustenia a nastavenia parametrov generovania [17].

Ollama umožňuje lokálne spúšťanie veľkých jazykových modelov. Takéto modely môžu generovať dialógy, opisy predmetov, pravidiel, naratívne prvky alebo štruktúrované výstupy, ktoré následne využívajú ďalšie časti aplikácie [18].

Prehľad vybraných nástrojov z pohľadu ich využitia v hernom vývoji je zhrnutý v tabuľke 4.

Tabuľka 4: Porovnanie existujúcich GenAI nástrojov z pohľadu herného vývoja

Nástroj	Typ využitia	Výstup	Spôsob integrácie
Unity AI	asistovaný vývoj v Unity editore	úpravy assetov, pomoc pri vývoji	integrácia priamo v prostredí Unity
Scenario	generovanie vizuálnych assetov	postavy, objekty, textúry	externá webová služba alebo API
Rosebud AI	prototypovanie hier z textu	herné prototypy a scény	webová aplikácia založená na textovom zadaní
Stable Diffusion WebUI	lokálne obrazové generovanie	obrázky, textúry, sprity	lokálny HTTP server s API rozhraním
Ollama	lokálne spúšťanie LLM	text, JSON, štruktúrované dáta	lokálny backend dostupný cez API

Z porovnania vyplýva, že GenAI nástroje možno do herného vývoja zapojiť rôznymi spôsobmi. Niektoré sú dostupné priamo vo vývojovom prostredí, iné fungujú ako webové služby alebo samostatné lokálne backendy. Pre navrhované riešenie je dôležitý najmä prístup založený na lokálnom HTTP rozhraní, pretože umožňuje oddeliť Unity aplikáciu od generatívnych modelov a zároveň ponechať kontrolu nad použitými nástrojmi a parametrami generovania.

3.5 Vstupy a výstupy generatívnych modelov

Pri práci s generatívnymi modelmi je dôležité rozlišovať medzi vstupom, ktorý model prijíma, výstupom, ktorý vytvára, a spôsobom, akým je tento výstup ďalej spracovaný. V hernom vývoji sa výstup generatívneho modelu zvyčajne nepoužíva priamo bez ďalších úprav. Často je potrebné overiť jeho formát, technickú použiteľnosť, vizuálnu kvalitu alebo súlad s požiadavkami konkrétnej hry.

Tabuľka 5: Vstupy a výstupy generatívnych modelov v hernom vývoji

Typ modelu	Vstup	Výstup
Jazykový model	textové zadanie, téma, systémový prompt	text, dialóg, opis, pravidlá alebo štruktúrované dáta
Textovo-obrazový model	prompt, negatívny prompt, parametre generovania	obrázok, textúra, ikona alebo sprite
Obrazovo-obrazový	referenčný obrázok, textové zadanie, negatívne zadanie, miera zachovania pôvodného obrázka	upravený alebo štylizovaný obrázok, variant existujúceho assetu alebo sprite
Zvukový model	textový opis, referenčný zvuk alebo štýl	hudba, zvukový efekt alebo hlas
3D generatívny model	textový opis, obrázok alebo referenčný tvar	3D objekt alebo geometria
Postprocessingová vrstva	výstup generatívneho modelu	upravený alebo technicky použiteľnejší výstup

3.6 Požiadavky na použiteľnosť generovaných assetov

Pri použití GenAI v hernom engine nestačí hodnotiť iba estetickú kvalitu výstupu. Obrázok, ktorý pôsobí vizuálne dobre ako samostatná ilustrácia, nemusí byť vhodný ako herný asset. Použiteľnosť závisí od toho, či výstup spĺňa technické a funkčné požiadavky konkrétnej hry.

Pri textúrach je dôležité, aby neobsahovali výraznú perspektívu, horizont, scénické kompozície alebo dominantné objekty, ktoré by narúšali ich aplikáciu na povrch. Textúra určená pre terén by mala pôsobiť ako povrchový materiál, nie ako samostatná krajinná fotografia. V opačnom prípade môže po aplikovaní na tilemap pôsobiť rušivo alebo nečitateľne.

Pri sprite assetoch je dôležité jasné oddelenie objektu od pozadia. Sprite by mal obsahovať jeden hlavný objekt, primerané okraje, vhodnú mierku a čitateľnú siluetu. Problémom bývajú najmä rámiky, text, pozadie podobnej farby ako objekt, deformácie alebo neúplné časti postavy. Takéto výstupy si vyžadujú postprocessing, napríklad odstránenie pozadia, orezanie alebo výber dominantnej komponenty.

Dôležitá je aj konzistentnosť štýlu medzi assetmi. Ak sú pozadie, terén, hráč a nepriatelia generovaní samostatne, môže sa stať, že vizuálne nebudú pôsobiť ako súčasť jedného sveta. Tento problém možno zmierniť tým, že všetky prompty vychádzajú z

jedného tematického plánu, obsahujú spoločnú paletu a používajú konzistentné štýlové obmedzenia.

3.7 Limity a riziká generatívneho prístupu

Generatívne modely prinášajú do vývoja hier nové možnosti, ale zároveň aj viaceré riziká. Jedným z hlavných problémov je nedeterministickosť výstupov. Rovnaký alebo podobný prompt môže pri opakovanom generovaní viesť k odlišným výsledkom. To môže byť výhodné pri tvorbe variácií, ale problematické pri potrebe konzistentného štýlu alebo reprodukovateľnosti.

Ďalším rizikom je nedodržanie technických požiadaviek. Obrazový model môže vytvoriť vizuálne pôsobivý obrázok, ktorý však nie je použiteľný ako textúra alebo sprite. Jazykový model môže zase vytvoriť text, ktorý nezodpovedá očakávanej dátovej štruktúre. Preto je potrebné doplniť generovanie o validačné mechanizmy.

Významným faktorom sú aj výpočtové nároky. Lokálne generovanie obrázkov môže byť pomalé a závisí od výkonu grafickej karty. Pri použití počas behu aplikácie je preto potrebné zvoliť takú architektúru, ktorá nebude blokovat hlavné vlákno Unity a umožní používateľovi priebežne sledovať stav generovania.

Z hľadiska návrhu systému sú preto dôležité štyri princípy: modularita, validácia, fallback mechanizmy a asynchrónne spracovanie. Modularita umožňuje nahradiť jazykový alebo obrazový backend bez zásadnej zmeny celej aplikácie. Validácia znižuje riziko spracovania chybných výstupov. Fallback mechanizmy zabezpečujú, že hra zostane funkčná aj pri zlyhaní AI vrstvy. Asynchrónne spracovanie znižuje negatívny vplyv generovania na používateľskú skúsenosť.

4 Návrh riešenia

Cieľom tejto kapitoly je navrhnúť riešenie, ktoré umožní využiť GenAI pri tvorbe obsahu v Unity projekte. Riešenie sa zameriava na spracovanie textového vstupu používateľa, vytvorenie tematického návrhu úrovne, generovanie vizuálnych assetov a ich následné použitie v 2D platformovej scéne. Návrh vychádza z teoretických poznatkov uvedených v predchádzajúcich kapitolách a prihliada na požiadavky hrateľnosti, stability a opakovateľnosti prototypu.

4.1 Ciele navrhovaného riešenia

Navrhované riešenie má overiť, akým spôsobom je možné zapojiť generatívne modely do procesu tvorby herného obsahu. Cieľom je vytvoriť prototyp pipeline, ktorá prepojí používateľský vstup, generovanie návrhu úrovne, tvorbu vizuálnych assetov a ich aplikáciu v Unity scéne. Pipeline by mala byť schopná:

- prijať textový vstup používateľa opisujúci tému hry alebo úrovne,
- spracovať vstup pomocou jazykového modelu,
- vytvoriť štruktúrovaný plán úrovne a podklady pre generovanie assetov,
- odoslať požiadavky do text-to-image modelu,
- prijať a spracovať vygenerované obrazové výstupy,
- upraviť textúry a sprity do použiteľnej podoby,
- aplikovať výsledné assety do procedurálne vytvorenej Unity scény,
- zachovať funkčnosť hry aj pri čiastočnom zlyhaní AI vrstvy.

4.2 Požiadavky na systém

Požiadavky na systém možno rozdeliť na funkčné, technické a používateľské. Funkčné požiadavky určujú, čo má systém vykonávať. Technické požiadavky určujú obmedzenia vyplývajúce z použitých technológií. Používateľské požiadavky súvisia s použiteľnosťou prototypu a spätnou väzbou počas generovania.

4.2.1 Funkčné požiadavky

Z funkčného hľadiska musí systém umožniť vytvoriť kompletnú tematickú úroveň v 2D platformovej hre. Používateľ zadá tému, napríklad futuristické mesto, lesné ruiny

alebo ladovú jaskyňu. Jazykový model následne vytvorí štruktúrovaný plán, ktorý obsahuje prompty pre jednotlivé typy assetov a doplnkové herné parametre. Systém musí generovať minimálne pozadie úrovne, textúru terénu pre plošiny, sprite hráča, sprity nepriateľov, sprite bossa, sprite projektilu, sprite zberateľného objektu, krátky naratívny alebo tematický opis úrovne a základné herné parametre ovplyvňujúce náročnosť.

4.2.2 Technické požiadavky

Z technického hľadiska musí byť systém navrhnutý tak, aby neblokoval hlavné vlákno Unity. Generovanie pomocou jazykových a obrazových modelov je výpočtovo náročné a môže trvať niekoľko sekúnd až minút. Priame blokujúce čakanie na odpoveď by spôsobovalo zamrznutie používateľského rozhrania a zhoršenie používateľskej skúsenosti. Komunikácia s backendmi preto musí prebiehať asynchrónne.

Ďalšou požiadavkou je modularita. Jazykový model, obrazový model a Unity časť musia byť oddelené tak, aby bolo možné v budúcnosti nahradiť jeden modul bez zásadnej zmeny ostatných častí systému. Napríklad Ollama možno nahradiť iným LLM runtime prostredím alebo A1111 možno nahradiť iným obrazovým backendom, ak bude zachované vhodné rozhranie.

Systém musí počítať aj s nevalidnými alebo nekvalitnými výstupmi. Jazykový model nemusí vždy vrátiť správne štruktúrovaný JSON a obrazový model môže vytvoriť nepoužiteľný obrázok. Preto je potrebné doplniť validáciu, opakovanie požiadaviek a fallback mechanizmy.

4.2.3 Používateľské požiadavky

Používateľ musí byť informovaný o stave generovania. Keďže jednotlivé kroky pipeline môžu trvať dlhší čas, používateľské rozhranie má priebežne zobrazovať aktuálny stav, napríklad generovanie plánu, tvorbu pozadia, tvorbu terénu alebo aplikáciu assetov do scény.

Z pohľadu hry je dôležité, aby úroveň zostala hrateľná aj v prípade, že výstup GenAI nebude použiteľný. Môže ísť napríklad o vizuálne nekvalitný asset, chybný formát výstupu, príliš dlhý čas generovania alebo úplné zlyhanie modelu. AI preto nemá byť jediným zdrojom funkčnosti úrovne. Základnú hrateľnosť musí zabezpečovať procedurálny generátor a fallback vrstvy.

4.3 Analýza možností nasadenia AI backendu

Pri návrhu bolo potrebné rozhodnúť, kde budú generatívne modely spustené. Uvažované boli tri základné možnosti: lokálne nasadenie, vzdialené API a hybridný prístup.

Lokálne nasadenie poskytuje plnú kontrolu nad modelmi, parametrami generovania a verziami použitých nástrojov. Zároveň odstraňuje závislosť od externých API služieb a umožňuje experimentovanie bez nutnosti internetového pripojenia. Nevýhodou sú vyššie nároky na konfiguráciu prostredia, dostupný hardvér a lokálne výpočtové zdroje.

Vzdialené API by zjednodušilo počiatočné nasadenie a znížilo požiadavky na lokálny výpočtový výkon, no za cenu priebežných nákladov, závislosti od internetu a potreby správy API kľúčov.

Hybridný prístup, kombinujúci lokálny backend s cloudovým fallbackom, bol vyhodnotený ako zaujímavý pre produkčné scenáre, avšak zbytočne by zvyšoval komplexitu prototypu.

Tabuľka 6: Porovnanie možností nasadenia AI backendu

Kritérium	Lokálne	Vzdialené API	Hybridné
Kontrola nad modelmi a verziami	vysoká	závislá od poskytovateľa	čiastočná
Závislosť od externých služieb	nízka	vysoká	stredná
Nároky na lokálny hardvér	vyššie	nižšie	stredné
Komplexita riešenia	stredná	nízka	vysoká

4.4 Voľba konkrétnych nástrojov

Na základe porovnania možností nasadenia bolo ako najvhodnejšie zvolené lokálne riešenie. Tento prístup umožňuje mať väčšiu kontrolu nad použitými modelmi, ich verziami a parametrami generovania. Zároveň znižuje závislosť od externých služieb a umožňuje jednoduchšie opakovanie experimentov v rovnakých podmienkach. Hoci lokálne nasadenie kladie vyššie nároky na konfiguráciu prostredia a dostupný hardvér, pre účely výskumného prototypu je vhodné, pretože poskytuje stabilné a kontrolované prostredie na testovanie navrhovaného riešenia.

Po rozhodnutí pre lokálne nasadenie nasledoval výber konkrétnych nástrojov pre jednotlivé vrstvy riešenia.

4.4.1 LLM vrstva

Pre lokálne spúšťanie jazykových modelov boli zvažované viaceré možnosti. *LM Studio* [19] ponúka pohodlné grafické rozhranie a jednoduchú prácu s modelmi. Jeho rozhranie je však orientované skôr na interaktívne použitie než na stabilnú programovú integráciu.

Priame použitie *llama.cpp* [20] by poskytlo vysokú mieru kontroly, no za cenu náročnejšej konfigurácie a väčšej implementačnej záťaže. Cloudové riešenia, ako napríklad OpenAI API, boli vylúčené už na úrovni požiadaviek.

Ako najvhodnejšie riešenie bolo zvolené *Ollama* [18], najmä vďaka jednoduchej inštalácii, stabilnému REST API a dobrej podpore lokálne prevádzkovaných modelov. Dôležitou výhodou je aj možnosť získavať štruktúrované JSON výstupy pri vhodne navrhnutom systémovom prompte, čo je pre následné spracovanie v Unity kľúčové.

4.4.2 Obrazová vrstva

Pre generovanie obrazových assetov boli analyzované nástroje *AUTOMATIC1111* [21], *ComfyUI* [22] a *InvokeAI* [23]. ComfyUI poskytuje vysokú flexibilitu pri tvorbe komplexných pracovných postupov. Jeho node-based prístup však predstavuje zbytočne rozsiahle riešenie pre jednoduché `txt2img` generovanie. InvokeAI využíva odlišnú štruktúru API a vyžadoval by dodatočnú integračnú vrstvu.

Ako najvhodnejší nástroj bolo preto zvolené rozhranie *AUTOMATIC1111*, ktoré poskytuje stabilné REST API, priamu podporu generovania pomocou endpointu `/sdapi/v1/txt2img` a dostatočnú kontrolu nad inferenčnými parametrami. Dôležitým faktorom bola aj široká dostupnosť checkpointov kompatibilných s týmto prostredím.

Tabuľka 7: Porovnanie zvažovaných nástrojov pre jednotlivé vrstvy systému

Nástroj	Typ	REST API	Lokálne	Výber
Ollama	LLM	áno	áno	✓
LM Studio	LLM	obmedzené	áno	
llama.cpp	LLM	manuálne	áno	
OpenAI API	LLM	áno	nie	
A1111	Image gen.	áno	áno	✓
ComfyUI	Image gen.	komplexné	áno	
InvokeAI	Image gen.	odlišné	áno	

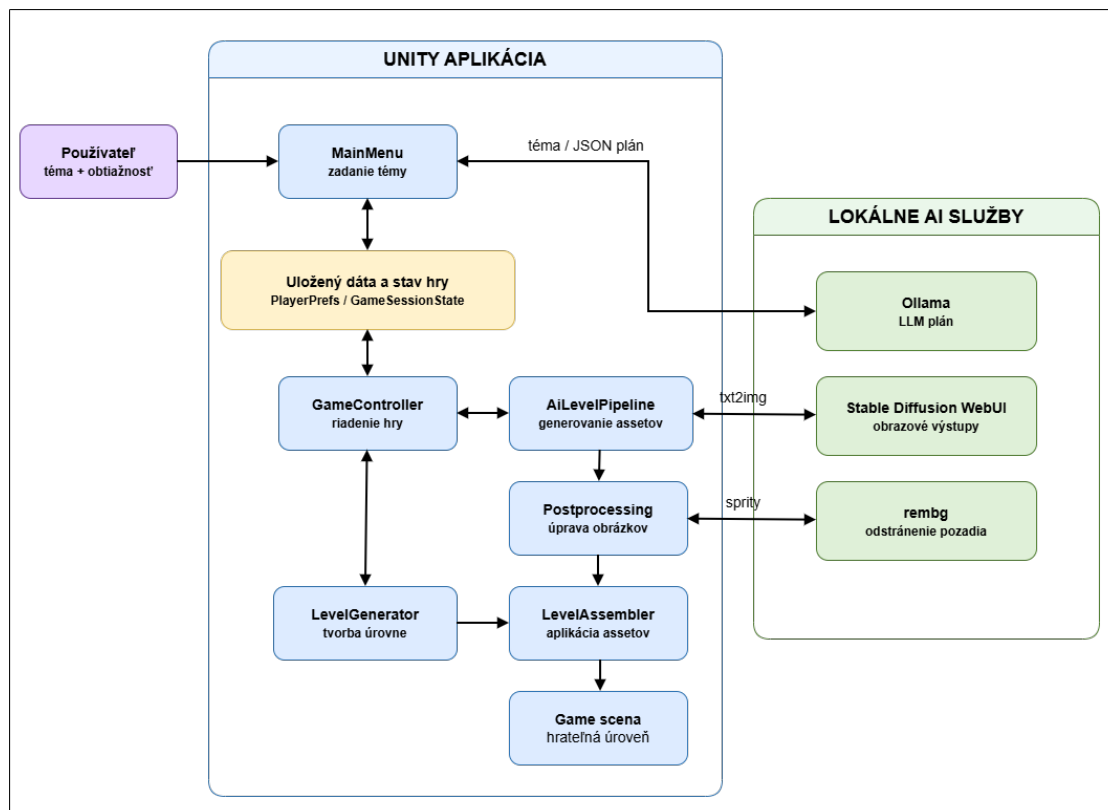
4.5 Architektúra systému

Navrhnutý systém je rozdelený do troch logicky oddelených modulov s jasne definovanými rozhraniami:

1. **LLM modul** - prijíma textový vstup od používateľa, komunikuje s Ollama serverom a vracia štruktúrovaný JSON plán generovania,

2. **Obrazový modul** - prijíma jednotlivé položky plánu, komunikuje so Stable Diffusion backendom a vracia dekódované obrazové dáta,
3. **Unity integračný modul** - riadi priebeh celej pipeline, spravuje stav používateľského rozhrania a aplikuje výsledné assety do scény.

Zjednodušená architektúra systému a základné komunikačné väzby medzi hlavnými časťami riešenia sú znázornené na obrázku 4.



Obr. 4: Zjednodušená architektúra systému a vzťahy medzi hlavnými modulmi.

Takéto rozdelenie zjednodušuje testovanie a zároveň umožňuje v budúcnosti nahradiť jednu vrstvu systému bez potreby zásadných zmien v ostatných moduloch.

4.5.1 LLM modul

LLM modul zabezpečuje komunikáciu s lokálne spusteným jazykovým modelom. Unity odosiela používateľskú tému a systémový prompt, ktorý definuje očakávanú štruktúru odpovede. Výstupom je JSON objekt, ktorý obsahuje položky potrebné pre ďalšie generovanie.

Dôležitou súčasťou LLM modulu je validácia odpovede. Model môže vygenerovať odpoveď, ktorá nie je platným JSON objektom, prípadne vynechá niektoré povinné

polia. Preto modul po prijatí odpovede kontroluje jej syntaktickú správnosť a prítomnosť povinných položiek. Ak odpoveď nie je použiteľná, systém môže zopakovať požiadavku alebo použiť fallback plán.

4.5.2 Obrazový modul

Obrazový modul zabezpečuje komunikáciu s obrazovým backendom. Pre každý typ assetu vytvára samostatnú požiadavku, ktorá obsahuje prompt, negatívny prompt a technické parametre generovania. Po prijatí odpovede dekoduje Base64 dáta do textúry a odovzdáva ich ďalším častiam systému.

Pri návrhu Obrazový modulu bolo potrebné rozlišovať medzi typmi výstupov. Pozadie, terénna textúra a sprite objekty majú rozdielne požiadavky. Pozadie môže byť scénické a vizuálne bohaté, zatiaľ čo terénna textúra musí pôsobiť ako povrchový materiál. Sprite objekt musí byť oddeliteľný od pozadia a použiteľný ako samostatná herná entita.

4.5.3 Unity integračný modul

Unity integračný modul prepája procedurálny generátor úrovne, AI backendy a hernú scénu. Riadi poradie jednotlivých krokov, spracováva chyby a zabezpečuje, aby sa výsledné assety aplikovali na správne objekty.

Tento modul zároveň uchováva stav generovania a zabraňuje duplicitnému spusteniu pipeline. Počas generovania aktualizuje používateľské rozhranie, aby používateľ videl, ktorá časť procesu práve prebieha.

4.6 Dátový kontrakt medzi LLM a Unity

Komunikácia medzi jazykovým modelom a Unity je založená na pevne definovanom dátovom kontrakte vo forme JSON správy. Cieľom kontraktu je zabezpečiť, aby výstup jazykového modelu nebol voľným textom, ale štruktúrovanými údajmi, ktoré je možné automaticky spracovať v Unity.

JSON správa obsahuje textové zadania pre generovanie jednotlivých vizuálnych prvkov úrovne, ich negatívne varianty a základné herné parametre. Vizuálna časť kontraktu opisuje najmä pozadie, terén, hráča, nepriateľov, bossa, projektily a zberateľné objekty. Doplnkové polia určujú spoločné vlastnosti prostredia, napríklad materiál, typ povrchu a farebnú paletu. Herné parametre následne umožňujú ovplyvniť vybrané vlastnosti nepriateľov a zberateľných objektov.

```
{  
  "bg": "", "bgNeg": "",
```

```

"ground": "", "groundNeg": "",
"player": "", "playerNeg": "",
"enemy": "", "enemyNeg": "",
"pickup": "", "pickupNeg": "",
"flyingEnemy": "", "flyingEnemyNeg": "",
"shootingEnemy": "", "shootingEnemyNeg": "",
"bossEnemy": "", "bossEnemyNeg": "",
"material": "", "surface": "", "palette": "",
"enemySpeed": 2.0,
"enemyDamage": 1,
"enemyPatrolRange": 3.0,
"pickupHealAmount": 0,
"pickupScoreValue": 50,
"enemyType": "ground"
}

```

Polia s príponou **Neg** predstavujú negatívne zadania pre obrazový model. Slúžia na obmedzenie nežiaducich vlastností výstupu, napríklad deformácií, textu v obrázku, nevhodného pozadia alebo nesprávnej perspektívy. Ostatné textové polia opisujú požadovaný vzhľad jednotlivých prvkov úrovne.

Použitie pevnej JSON štruktúry zvyšuje spoľahlivosť spracovania výstupu. Unity nemusí interpretovať prirodzený jazyk, ale pracuje s očakávanými poľami. V implementácii je preto jazykový model inštruovaný, aby vrátil iba minifikovaný JSON bez sprievodného textu, formátovania Markdown alebo dodatočného komentára. Takýto prístup znižuje riziko chyby pri parsovaní a vytvára jasnú hranicu medzi LLM vrstvou a Unity časťou systému.

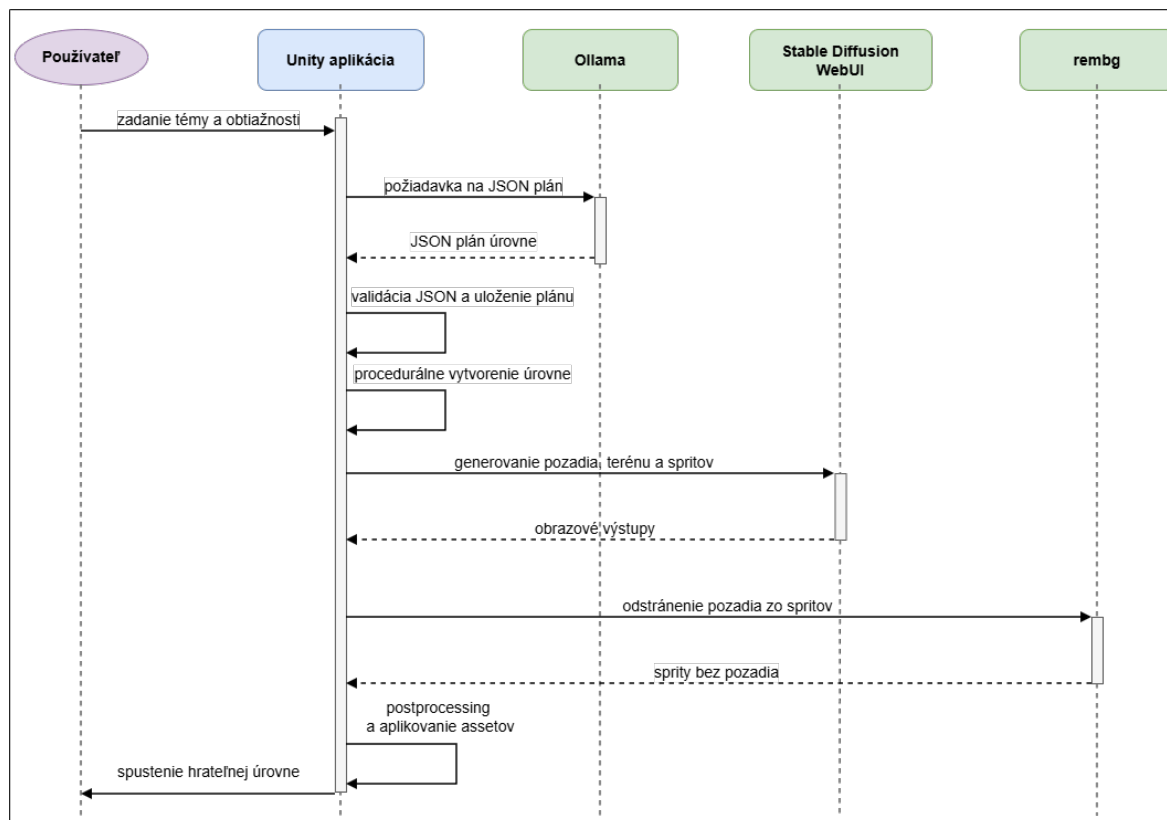
4.7 Sekvenčné spracovanie pipeline

Generovanie assetov je navrhnuté ako sekvenčný proces. Jednotlivé výstupy sa negenerujú paralelne, ale postupne. Takýto prístup je vhodný najmä pri lokálnom generovaní, kde je potrebné počítať s obmedzenými výpočtovými zdrojmi a rizikom preťaženia grafickej pamäte. Zároveň zjednodušuje kontrolu jednotlivých krokov a umožňuje lepšie určiť miesto prípadného zlyhania.

Proces je rozdelený medzi dve scény. V scéne *MainMenu* používateľ zadá tému a zvolí obtiažnosť. Na základe týchto údajov jazykový model vytvorí štruktúrovaný JSON plán, ktorý sa uloží a následne použije v hernej scéne *Game*. V tejto scéne sa vytvorí

procedurálna úroveň, vygenerujú sa vizuálne výstupy a po spracovaní sa aplikujú do herného prostredia.

Sekvenčný priebeh generovania je znázornený na obrázku 5.



Obr. 5: Sekvenčný diagram hlavného priebehu generovania obsahu.

Hlavné kroky procesu možno zhrnúť nasledovne:

1. zadanie témy a obtiažnosti používateľom,
2. vytvorenie, validácia a uloženie JSON plánu,
3. načítanie plánu v hernej scéne,
4. procedurálne vytvorenie geometrie úrovne,
5. generovanie pozadia, terénu, postavy, nepriateľov a doplnkových objektov,
6. odstránenie pozadia a následné spracovanie sprite výstupov,
7. aplikácia výsledných assetov do Unity scény,
8. spustenie hernej logiky.

Súčasťou procesu sú aj náhradné mechanizmy. Ak sa niektorý výstup nepodarí vygenerovať alebo nie je použiteľný, systém môže použiť náhradný plán alebo predvolený asset. Vďaka tomu zostáva základná hrateľnosť úrovne zachovaná aj pri čiastočnom zlyhaní AI vrstvy.

Pri návrhu sa zároveň počíta s tým, že hra môže obsahovať viacero úrovní v rámci jedného priebežného spustenia. Z tohto dôvodu môže byť časť dát pre nasledujúcu úroveň pripravená počas hrania aktuálnej úrovne, čím sa znižuje čakanie používateľa pri prechode medzi scénami.

4.8 Riziká návrhu a navrhnuté opatrenia

Pri návrhu boli identifikované viaceré riziká. Najvýznamnejšie súvisia s nepredvídateľnosťou generatívnych modelov, kvalitou obrazových výstupov, stabilitou lokálnych backendov a výpočtovou náročnosťou.

Tabuľka 8: Identifikované riziká návrhu a navrhnuté opatrenia

Riziko	Možný dôsledok	Navrhnuté opatrenie
Nevalidný JSON výstup z LLM	Nemožnosť spracovať plán v Unity	Validácia, extrakcia JSON časti, opakovanie požiadavky, fallback plán
Nevhodná terénna textúra	Vizuálne rušivá alebo nepoužiteľná úroveň	Negatívne prompty, validačné pravidlá, fallback tile
Sprite s pozadím alebo rámom	Nečitateľný objekt v scéne	Odstránenie pozadia pomocou rembg, náhradné color-key spracovanie, orezanie a úprava alfa kanála
Pretaženie GPU	Zlyhanie generovania alebo pád backendu	Sekvenčné generovanie, obmedzené rozlíšenie, kontrolované parametre
Nekonzistentný vizuálny štýl	Assety nepôsobia ako súčasť jedného sveta	Spoločný tematický plán, paleta, jednotné prompt šablóny
Výpadok backendu	Nedokončené generovanie úrovne	Chybová signalizácia, fallback plán, procedurálne náhradné assety

4.9 Zhrnutie návrhu

Navrhnuté riešenie prepája procedurálne generovanie úrovne v Unity s lokálne prevádzkovanou GenAI. Jazykový model slúži ako plánovacia vrstva, ktorá z používateľského vstupu vytvára štruktúrovaný plán. Obrazový model generuje vizuálne assety na základe promptov z tohto plánu. Unity zabezpečuje orchestráciu, validáciu, postprocessing a aplikovanie výsledkov do hernej scény.

Kľúčovými návrhovými rozhodnutiami sú lokálne nasadenie backendov, sekvenčné spracovanie požiadaviek a zavedenie fallback mechanizmov. Výber nástrojov bol motivovaný lokálnou prevádzkyschopnosťou, stabilitou API a primeranou zložitou integrácie. Takýto návrh umožňuje vytvoriť funkčný prototyp, ktorý demonštruje možnosti aj obmedzenia využitia GenAI v prostredí Unity.

5 Implementácia riešenia

Implementácia navrhnutého riešenia prebiehala iteratívne. Počas vývoja sa ukázalo, že dôležitou časťou práce nebolo iba samotné prepojenie Unity s AI backendom, ale najmä zabezpečenie stabilného spracovania výstupov generatívnych modelov. Z tohto dôvodu bolo potrebné doplniť validačné kroky, spracovanie chybových stavov a fallback mechanizmy, ktoré umožňujú zachovať funkčnosť hry aj v prípade neúplného alebo nepoužiteľného výstupu AI.

Cieľom tejto kapitoly je opísať priebeh implementácie, postupnosť hlavných rozhodnutí a problémy, ktoré bolo potrebné riešiť pri prepájaní jazykového modelu, obrazového modelu a Unity scény.

5.1 Východiskový stav a implementačný cieľ

Implementácia vychádzala z návrhu systému, ktorého cieľom bolo prepojiť procedurálne generovanie úrovne v Unity s výstupmi GenAI. Výsledný prototyp mal umožniť spracovanie textového vstupu používateľa, vytvorenie štruktúrovaného plánu pomocou jazykového modelu, generovanie obrazových assetov a ich následné začlenenie do 2D platformovej scény.

Cieľom implementácie nebolo iba overiť samotné API volania medzi jednotlivými časťami systému, ale aj praktickú použiteľnosť vygenerovaných výstupov v hernom prostredí. Počas vývoja sa preto priebežne kontrolovalo, či úroveň zostáva hrateľná, či sú textúry vizuálne použiteľné a či sprity fungujú ako samostatné objekty so správnym zobrazením a správaním v scéne. Z tohto dôvodu bol zvolený inkrementálny postup, pri ktorom sa po každej väčšej etape testovali výsledky priamo v hre.

5.2 Štruktúra Unity projektu

Projekt bol implementovaný ako dvojscénová Unity aplikácia. Prvá scéna, *MainMenu*, slúži na zadanie témy hry, výber obtiažnosti a spustenie generovania plánu pomocou jazykového modelu. Výsledný plán sa následne uloží a použije ako vstup pre hernú scénu. Súčasťou hlavného menu je aj informačné okno, ktoré používateľovi stručne vysvetľuje, ktoré časti hry sú generované pomocou AI a v akom poradí tento proces prebieha. Druhá scéna, *Game*, predstavuje samotné herné prostredie. V tejto scéne sa vytvára procedurálne rozloženie úrovne, načítavajú sa údaje pripravené v hlavnom menu a aplikujú sa vygenerované vizuálne assety. Scéna zároveň obsahuje hernú logiku, fyziku, prácu s hráčom, nepriateľmi, zberateľnými objektmi a prechodmi medzi úrovňami.

Prenos základných údajov medzi scénami je riešený pomocou *PlayerPrefs*. Táto časť uchováva najmä údaje potrebné na spustenie hernej scény, napríklad zadanú tému a plán úrovne. Priebežný stav hry je oddelený do triedy *GameSessionState*, ktorá uchováva hodnoty platné počas jedného priebežného hrania. Patrí sem skóre, počet životov, aktuálny index úrovne, seed, zvolená obtiažnosť, pripravené dáta pre nasledujúcu úroveň a uzamknutý vzhľad hráča.

Takéto rozdelenie oddeľuje dáta potrebné na prenos medzi scénami od stavu hry počas jej behu. Vďaka tomu je možné pri prechodoch medzi úrovňami zachovať priebežný stav hráča, zvolenú náročnosť aj rovnaký vzhľad postavy. Zároveň sa tým znižuje riziko, že sa do dlhodobu ukladaných údajov dostanú hodnoty, ktoré majú význam iba počas aktuálneho spustenia hry.

Pre lepší prehľad sú hlavné triedy a komponenty implementovaného prototypu zhrnuté v tabuľke 9.

Tabuľka 9: Zodpovednosti hlavných tried implementovaného prototypu

Trieda	Zodpovednosť
<i>MainMenuController</i>	Spracovanie témy, obtiažnosti a spustenie tvorby plánu.
<i>OllamaClient</i>	Komunikácia s jazykovým modelom a získanie JSON plánu.
<i>LevelPlan</i>	Dátová reprezentácia plánu úrovne, promptov a herných parametrov.
<i>GameController</i>	Riadenie priebehu hry, načítanie plánu a prechody medzi úrovňami.
<i>AILevelPipeline</i>	Generovanie obrazových výstupov a koordinácia ich spracovania.
<i>LevelGenerator</i>	Procedurálne vytvorenie rozloženia úrovne a herných prvkov.
<i>LevelAssembler</i>	Aplikovanie textúr a spritov do Unity scény.
<i>RembgClient</i>	Odstránenie pozadia zo spritov pomocou nástroja <i>rembg</i> .
<i>TextureUtils</i>	Náhradné čistenie obrázkov, orezanie a úprava alfa kanála.
<i>GameSessionState</i>	Uchovanie priebežného stavu hry počas jedného spustenia.

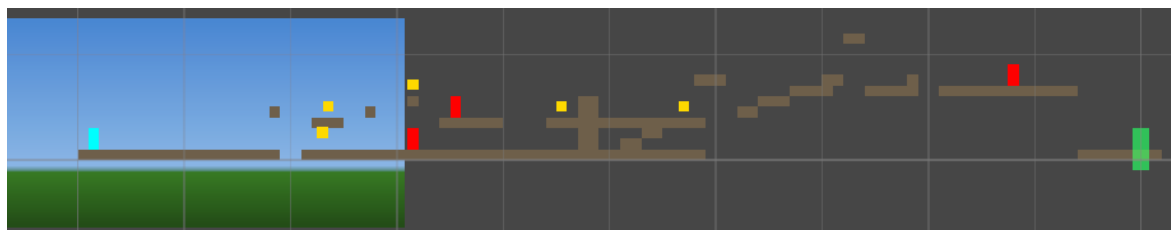
5.3 Procedurálne generovanie úrovne

Po vytvorení základnej štruktúry projektu nasledovala implementácia modulu na procedurálne generovanie úrovne. Jeho úlohou bolo vytvoriť základnú 2D platformovú scénu ešte pred aplikovaním finálnych vizuálnych assetov. Tento krok zabezpečuje rozmiestnenie terénu, plošín, nepriateľov, zberateľných objektov a výstupu z úrovne.

Trieda *LevelGenerator* vytvára segmenty zeme, medzery medzi plošinami, plávajúce plošiny a výškové zmeny. Pri generovaní sa zohľadňujú pohybové možnosti hráča, aby bola výsledná úroveň priechodná. Geometria úrovne je reprezentovaná pomocou *tilemap*, ktorá sa neskôr využíva aj pri aplikovaní vygenerovaných textúr.

Výstupom tejto etapy bola funkčná úroveň bez finálnej vizuálnej štylizácie. Takto pripravený základ umožnil oddeliť hernú logiku od AI generovaných assetov, čo je dôležité najmä v prípade, že by niektorý vizuálny výstup nebol použiteľný.

Generátor bol následne rozšírený o údaje odvodené z AI plánu. Tie ovplyvňujú najmä typ nepriateľov, zvolenú obtiažnosť, ich hustotu a celkovú náročnosť úrovne. Na poslednej úrovni sa navyše vytvára boss aréna s väčším voľným priestorom a pevným podkladom. Tým sa procedurálne generovanie prepojilo s plánom vytvoreným jazykovým modelom.



Obr. 6: Výsledok procedurálneho generovania platformového rozloženia.

5.4 Orchestrácia generovacej pipeline

Generovanie AI obsahu bolo implementované ako sekvenčný asynchrónny proces, v ktorom jednotlivé kroky na seba nadväzujú v pevne určenom poradí. Plán úrovne sa vytvára už v scéne *MainMenu* pomocou triedy *OllamaClient* a následne sa uloží do *PlayerPrefs*. Po načítaní hernej scény *Game* trieda *AiLevelPipeline* tento plán spracuje a spustí generovanie obrazových assetov.

Poradie generovania je nasledovné:

1. pozadie úrovne,
2. textúra terénu,
3. hlavná postava,
4. pozemný nepriateľ,
5. lietajúci nepriateľ,
6. strieľajúci nepriateľ,
7. boss entita, iba na poslednom úrovni,
8. projektil,
9. zberateľný objekt.

Úpravy vygenerovaných sprite assetov, ako odstránenie pozadia pomocou color-key, orezanie obrázka a spracovanie alfa kanála, prebiehajú priebežne po vygenerovaní konkrétneho assetu. Netvoria preto samostatnú fázu pipeline, ale sú súčasťou jednotlivých požiadaviek na generovanie sprite objektov.

Riadenie procesu je realizované pomocou korutín a interných kontrolných podmienok, napríklad *IsGenerating*, ktoré zabráňujú opakovanému spusteniu generovania. Každý krok vracia výsledok spracovania, na základe ktorého sa okamžite rozhoduje o ďalšom postupe. Tento prístup zjednodušuje ladenie, pretože umožňuje presne určiť, v ktorom kroku došlo k zlyhaniu. Rovnaká vrstva zároveň zabezpečuje aj odovzdanie pripraveného balíka assetov pre nasledujúcu úroveň, čím sa znižuje potreba opakovaného čakania pri prechode medzi úrovňami.

5.5 Integrácia Ollama a generovanie štruktúrovaného plánu

Po vytvorení základnej generovacej pipeline nasledovala integrácia nástroja Ollama. Jeho úlohou bolo spracovať krátky textový vstup používateľa a vytvoriť z neho štruktúrovaný plán úrovne vo formáte JSON. Tento plán následne slúžil ako vstup pre ďalšie časti systému, najmä pre generovanie vizuálnych assetov a nastavenie vybraných herných parametrov.

5.5.1 Voľba modelu

V počiatočnej fáze bol testovaný model *Llama 3.2*. Pri praktickom použití sa však ukázalo, že jeho výstupy neboli vždy vhodné pre striktné definovanú JSON schému. Model v niektorých prípadoch dopĺňal sprievodný text, menil názvy polí alebo vynechával povinné položky. Takéto odchýlky spôsobovali problémy pri následnom spracovaní odpovede v Unity.

Z tohto dôvodu bol ako vhodnejšia alternatíva zvolený model *Mistral 7B*[24]. Pri testovaní poskytoval stabilnejšie štruktúrované odpovede a lepšie dodržiaval požadovaný formát. Spoľahlivosť výstupu bola podporená aj použitím parametra `format = "json"`, ktorý obmedzuje odpoveď modelu na JSON formát a znižuje pravdepodobnosť nevalidného výstupu.

5.5.2 Komunikácia s API a validácia výstupu

Komunikácia medzi Unity a Ollama prebieha prostredníctvom HTTP požiadavky typu POST na rozhranie `/api/chat`. Každá požiadavka obsahuje systémovú správu s definíciou očakávanej štruktúry a používateľskú správu so zadanou témou. Do požiadavky

sa zároveň prenáša aj informácia o poradí aktuálnej úrovne, aby bolo možné prispôbiť výsledný plán fáze hry.

Požiadavky sú odosielané s nízkou hodnotou teploty (0.1), vypnutým streamovaním, nastaveným JSON formátom a obmedzeným počtom výstupných tokenov. Napriek týmto obmedzeniam nebolo možné predpokladať, že výstup modelu bude vždy úplný a správne spracovateľný. Preto bola do implementácie doplnená validačná vrstva, ktorá kontroluje prítomnosť povinných polí, extrahuje JSON časť odpovede a overuje jej použiteľnosť pred odovzdaním ďalším častiam systému.

Ak výstup jazykového modelu nie je použiteľný, systém automaticky vytvorí náhradný plán pomocou `FallbackPlan`. Tento plán vychádza zo zadanej témy a interných pravidiel, vďaka čomu môže hra pokračovať aj v prípade chyby alebo výpadku LLM vrstvy. Fallback mechanizmus tak slúži ako ochranná vrstva, ktorá zabezpečuje stabilitu prototypu nezávisle od kvality konkrétnej odpovede modelu.

5.6 Integrácia Stable Diffusion a generovanie obrazových assetov

Po vytvorení štruktúrovaného plánu nasledovala integrácia obrazového modelu prostredníctvom lokálne spusteného A1111 servera. Úlohou tejto časti bolo previesť textové prompty z plánu na obrazové výstupy, ktoré bolo možné následne spracovať a použiť v Unity scéne.

Pre jednotlivé typy assetov sa vytvára požiadavka typu `A1111Txt2ImgRequest`, ktorá sa odosiela na endpoint `/sdapi/v1/txt2img`. Základný prompt pochádza z výstupu jazykového modelu, pred odoslaním sa však dopĺňa o technické pokyny podľa typu assetu. Iné požiadavky sú kladené na pozadie, iné na textúru terénu a iné na samostatné sprite objekty, ako hráč, nepriateľ alebo zberateľný objekt. Odpoveď servera obsahuje obrazové dáta vo formáte Base64, ktoré sa po prijatí dekodujú a pripravujú na ďalšie spracovanie v Unity.

Pri generovaní sa ukázalo, že rôzne typy assetov vyžadujú odlišné nastavenie promptov a validácie. Pri teréne bolo potrebné obmedziť generovanie perspektívnych alebo scénických výstupov a smerovať model skôr k povrchovej textúre. Pri sprite objektoch bolo dôležité potlačiť vznik rámov, kariet, zložitých pozadí a neúplne oddelených objektov. Problémy spôsobovalo aj zapnuté nastavenie `tiling`, ktoré pri niektorých výstupoch vytváralo viditeľné opakovacie artefakty.

Výsledné riešenie preto kombinuje upravené prompty, negatívne prompty, vypnutie nevhodných nastavení na strane A1111 a viacstupňové spracovanie výstupov v Unity.

Tento postup umožnil zvýšiť použiteľnosť vygenerovaných assetov a pripraviť ich na následné vloženie do hernej scény.

5.7 Voľba checkpointu Stable Diffusion

Výber checkpointu Stable Diffusion výrazne ovplyvnil podobu generovaných assetov. Počas testovania sa ukázalo, že samotný checkpoint má na výsledný vizuálny štýl a konzistentnosť väčší vplyv než menšie úpravy promptov alebo inferenčných parametrov.

Na začiatku bol použitý základný model *Stable Diffusion 1.5*, ktorý slúžil najmä na overenie funkčnosti generovacej pipeline. Výstupy boli použiteľné na technické testovanie, avšak pri spritoch aj textúrach pôsobili pomerne všeobecne a menej výrazne. Následne bol testovaný checkpoint *Pixel Art XL*, ktorý dosahoval lepšie výsledky pri samostatných sprite objektoch, ale pri generovaní povrchových textúr bol príliš štylizovaný. Zvažovaný bol aj model *DreamShaper XL*, ktorý poskytoval kvalitnejšie výstupy, no pri lokálnom nasadení bol výrazne náročnejší na výkon a pamäť grafickej karty.

Ako finálny checkpoint bol zvolený *DreamShaper 8*. V rámci testovania predstavoval najvhodnejší kompromis medzi vizuálnou kvalitou, konzistentnosťou štýlu a hardvérovou náročnosťou. Zároveň poskytoval dostatočne použiteľné výsledky pre sprity aj povrchové textúry, čo bolo dôležité pre jednotné vizuálne doplnenie hernej scény.

Tabuľka 10: Porovnanie testovaných checkpointov Stable Diffusion

Checkpoint	Kvalita spritov	Kvalita textúr	Náročnosť
Stable Diffusion 1.5	stredná	stredná	nízka
Pixel Art XL	dobrá	slabšia	vyššia
DreamShaper XL	vysoká	vysoká	vysoká
DreamShaper 8	dobrá	dobrá	prijateľná

5.8 Postprocessing vygenerovaných assetov

Surový výstup obrazového modelu nie je vždy priamo použiteľný v Unity. Z tohto dôvodu bola do riešenia doplnená vrstva postprocessingu, ktorá pripravuje vygenerované textúry a sprity podľa ich účelu v hernej scéne.

Pri terénnych textúrach sa spracovanie vykonáva v triede `LevelAssembler`. Textúra sa pred použitím oreže a skontroluje, či je vhodná na aplikovanie na povrch úrovne. Ak výstup obsahuje výrazné artefakty, scénický pohľad alebo iný nevhodný obsah, nepoužije sa priamo. Namiesto neho sa vytvorí tematický fallback tile, ktorý zachová vizuálne prepojenie s témou úrovne a zároveň neohrozí funkčnosť scény.

Pri spritoch hráča, nepriateľov a zberateľných objektov bolo potrebné odstrániť pozadie tak, aby mohli byť použité ako samostatné objekty v Unity scéne. Primárne sa na tento účel používal lokálne spustený *rembg* server, s ktorým Unity komunikuje prostredníctvom triedy *RembgClient*. Tento nástroj využíva model na odstraňovanie pozadia a vracia obrázok s alfa kanálom.

Ak odstránenie pozadia pomocou *rembg* nebolo dostupné alebo neposkytlo použiteľný výsledok, systém mohol použiť náhradné spracovanie v triede *TextureUtils*. Tá zabezpečuje jednoduchšie odstránenie pozadia pomocou farebného kľúča, čistenie rohov obrázka, orezanie na relevantné pixely a úpravu alfa kanála.

5.9 Experiment s generovaním animácií

Počas implementácie bola overovaná aj možnosť generovať animácie priamo pomocou Stable Diffusion. Testované boli dva prístupy. Prvým bolo generovanie jednoduchého spritesheetu, teda viacerých snímok pohybu v jednom výstupe. Druhým bolo použitie režimu *img2img*, pri ktorom sa z jedného referenčného obrázka postupne vytvárali ďalšie pózy postavy.

Pri oboch prístupoch sa ukázalo, že výstupy nie sú dostatočne konzistentné pre použitie v platformovej hre. Medzi jednotlivými snímkami sa menili proporcie postavy, tvar hlavy, detaily oblečenia aj celková silueta. Pri statickom sprite môže byť takáto odchýlka prijateľná, no pri animácii spôsobuje viditeľné skoky medzi snímkami. Pri spritesheete sa navyše jednotlivé snímky často líšili mierkou a zarovnaním, čo sťažovalo ich priame použitie v Unity. Z týchto dôvodov nebolo generovanie animácií pomocou Stable Diffusion zaradené do finálneho riešenia. Namiesto toho sa použil jednoduchší prístup, pri ktorom sa animácia rieši procedurálne nad jedným statickým sprite assetom. Tento postup je stabilnejší, lepšie kontrolovateľný a postačuje pre potreby funkčného platformového prototypu.



Obr. 7: Ukážka nekonzistentného spritesheetu.



(a) Pôvodná referenčná postava.



(b) Výstup po úprave pomocou `img2img`.

Obr. 8: Porovnanie pôvodnej referenčnej postavy a výstupu získaného pomocou režimu `img2img`.

5.10 Aplikácia assetov do scény a herná vrstva

Po spracovaní vygenerovaných assetov nasleduje ich aplikácia do hernej scény. Terénna textúra sa pomocou triedy `LevelAssembler` aplikuje na bunky `terrainTilemap`, ktoré tvoria povrch úrovne. Tým sa procedurálne vytvorená geometria vizuálne doplní podľa témy určenej používateľom a spracovanej AI vrstvou.

Pri nepriateľských assetoch bolo potrebné rozlišovať viacero typov promptov podľa typu nepriateľa. Pozemný, lietajúci a strieľajúci nepriateľ, ako aj boss entita, majú odlišné vizuálne a funkčné požiadavky. Oddelené prompt vetvy umožnili vytvárať rôznorodejšie výstupy bez potreby manuálne pripravovať samostatné assety pre každý typ nepriateľa.

Na vizuálnu vrstvu nadväzuje herná logika. Jednotlivé assety sa nepoužívajú iba ako obrázky, ale sú priradené ku konkrétnym herným objektom s vlastným správaním. Nepriatelia používajú príslušné pohybové a útočné mechaniky, zberateľné objekty ovplyvňujú zdravie alebo skóre hráča a boss má samostatný health bar aj vlastný priebeh finálneho stretnutia.

Súčasťou hernej vrstvy je aj používateľské rozhranie. HUD zobrazuje aktuálne životy a skóre. Po ukončení hry sa zobrazí obrazovka výhry alebo prehry s finálnym skóre, cieľom odvodeným z príbehového kontextu a uloženým najvyšším skóre. Tieto mechanizmy začleňujú generovaný obsah do zrozumiteľného herného cyklu a umožňujú hráčovi jasne vnímať výsledok celého priebehu hry.

5.11 Stabilizácia pipeline a ladenie parametrov

V závere implementácie prebiehalo priebežné ladenie parametrov generovania a stabilizácia celej generovacej pipeline. Dôraz bol kladený najmä na zlepšenie konzistencie medzi jednotlivými typmi assetov, obmedzenie nežiaducich výstupov a spoľahlivé sekvenčné spracovanie požiadaviek. Medzi najčastejšie problémy patrili perspektívne deformácie, scénické výstupy namiesto povrchových textúr, vizuálne artefakty a nedostatočné oddelenie sprite objektov od pozadia.

Tabuľka 11: Zvolená konfigurácia parametrov generovania

Typ assetu	Kroky	CFG	Sampler	Tiling
Pozadie	20	7.0	DPM++ 2M Karras	nie
Terén	20-28	6.5-8.2	DPM++ 2M Karras	nie
Hráč, nepriateľ	20-22	9.0-9.5	DPM++ 2M Karras	nie
Zberateľný objekt	15-20	9.0	DPM++ 2M Karras	nie

Pri pozadí a terénnych textúrach sa osvedčili nižšie hodnoty CFG, pretože umožňovali prirodzenejšie vizuálne výstupy a menšiu náchylnosť na príliš výrazné artefakty. Pri sprite assetoch boli použité vyššie hodnoty CFG, keďže bolo potrebné presnejšie dodržanie promptu, jednoduchšie oddelenie objektu od pozadia a jasnejšia silueta postavy alebo nepriateľa.

5.12 Zhrnutie implementácie

Implementácia riešenia postupovala od prípravy základnej štruktúry Unity projektu, cez procedurálne generovanie 2D platformovej úrovne, až po zapojenie GenAI vrstvy. Tá prepája lokálny jazykový model s lokálne spusteným obrazovým modelom a umožňuje vytvárať vizuálne assety na základe textového zadania používateľa.

Za kľúčové možno považovať tri zistenia. Prvým je potreba spoľahlivého štruktúrovaného výstupu z jazykového modelu a náhradného plánu pre prípad nevalidnej odpovede. Druhým je význam voľby checkpointu Stable Diffusion, pri ktorom sa ako najvhodnejšie riešenie osvedčil model *DreamShaper 8*. Tretím je skutočnosť, že najväčšia časť implementačnej náročnosti nespočíva iba v samotnom generovaní obsahu, ale najmä v jeho validácii, opakovaní neúspešných pokusov a korektnom začlenení výstupov do hernej scény.

Výsledný systém preto nie je len jednoduchým generátorom obrázkov pre Unity. Ide o viacvrstvé riešenie, v ktorom jazyková vrstva vytvára tematický a herný kontext,

obrazová vrstva produkuje vizuálne assety a Unity vrstva zabezpečuje ich spracovanie, aplikáciu a prepojenie s hernou logikou konkrétnej úrovne.

6 Testovanie a vyhodnotenie

Cieľom testovania bolo overiť, či implementovaný prototyp dokáže na základe textového vstupu používateľa vytvoriť hrateľný 2D platformovú úroveň s vizuálnymi assetmi generovanými pomocou lokálne spustených modelov GenAI. Testovanie sa zameriavalo najmä na funkčnosť generovacej pipeline, úspešnosť generovania jednotlivých typov assetov a časovú náročnosť prípravy úrovne.

Dôležitou súčasťou testovania bolo aj sledovanie fallback mechanizmov. Tie sa aktivujú v prípadoch, keď niektorý výstup AI vrstvy nie je dostupný alebo nie je možné ho použiť v hernej scéne. Cieľom preto nebolo hodnotiť iba to, či sa podarilo vygenerovať obrazový výstup, ale aj to, či systém dokáže pokračovať v behu a pripraviť použiteľnú úroveň aj pri čiastočnom zlyhaní generovania.

6.1 Testovacie prostredie

Testovanie prebiehalo na lokálnom zariadení, na ktorom boli spustené všetky časti systému. Unity projekt, jazykový model aj nástroj na generovanie obrazových výstupov teda bežali lokálne bez využitia komerčných vzdialených služieb.

Tabuľka 12: Testovacie prostredie

Parameter	Hodnota
Operačný systém	Windows 11
Procesor	AMD Ryzen 7 7735HS with Radeon Graphics, 3.20 GHz
Operačná pamäť	16 GB
Grafická karta	NVIDIA GeForce RTX 4060 Laptop GPU, 8 GB VRAM
Verzia Unity	Unity 6.3 LTS (6000.3.5f2)
LLM runtime	Ollama
Použitý LLM model	<code>mistral:7b</code>
Nástroj na generovanie obrazových výstupov	Stable Diffusion WebUI AUTOMATIC1111
Checkpoint Stable Diffusion	<code>dreamshaper_8</code>
Sampler	DPM++ 2M Karras
Rozlíšenie generovaných obrázkov	1024 × 512 pre pozadie, 512 × 512 pre väčšinu herných objektov, 256 × 256 pre vybrané menšie objekty

6.2 Metodika testovania

Testovanie bolo rozdelené na funkčné a časové vyhodnotenie. Každá téma z testovacej sady bola spustená s rovnakými základnými nastaveniami generovania. Pri každom behu sa sledovalo, či sa podarilo vytvoriť plán úrovne, vygenerovať požadované assety, spracovať ich do použiteľnej podoby a pripraviť výstupný balík pre Unity scénu.

Za úspešný výstup sa považoval taký výstup, ktorý bol dostupný vygenerovanom balíku assetov a bolo možné ho ďalej použiť v hernej scéne. Ako fallback bol označený prípad, keď sa pôvodný výstup nepodarilo použiť a systém namiesto neho použil náhradný plán, náhradnú textúru alebo predvolený asset. Celá pipeline bola považovaná za úspešnú vtedy, keď generovanie skončilo bez kritickej chyby a systém vrátil použiteľný balík výstupov.

Pri časovom meraní sa sledovali hlavné kroky generovania: vytvorenie LLM plánu, generovanie pozadia, terénnej textúry, sprite assetov a celkové trvanie prípravy jednej úrovne. Meranie zahŕňalo čas potrebný na komunikáciu s backendom, získanie odpovede, dekodovanie výstupu a následné spracovanie v Unity.

6.3 Testovacia sada

Pre testovanie bola zvolená sada desiatich tém, ktoré pokrývajú rôzne typy prostredí a vizuálnych štýlov. Cieľom bolo overiť, či generovacia pipeline dokáže reagovať na prírodné, technické, fantasy, industriálne aj výrazne štylizované zadania.

Tabuľka 13: Testovacia sada tém

ID	Téma	Očakávaný vizuálny charakter
T1	Cyberpunk city	neónové mesto, technické prvky, tmavé pozadie
T2	Enchanted forest	les, prírodné farby, magické prvky
T3	Ice cave	ľad, modré odtiene, jaskynné prostredie
T4	Desert ruins	piesok, ruiny, staroveká architektúra
T5	Alien planet	mimozemské prostredie, netradičné farby
T6	Haunted castle	temný hrad, gotický štýl
T7	Volcanic dungeon	láva, skaly, tmavé červené odtiene
T8	Underwater temple	voda, tyrkysové farby, chrámové prvky
T9	Candy world	farebný štylizovaný svet
T10	Robot factory	kov, stroje, industriálne prostredie

6.4 Vyhodnotenie funkčnosti generovacej pipeline

Funkčné testovanie sledovalo, či jednotlivé časti generovacej pipeline dokážu vytvoriť použiteľné výstupy a či sa tieto výstupy následne správne spracujú. Testovanie bolo vykonané na sade desiatich tém, pričom každá téma bola spustená trikrát. Celkovo teda bolo vykonaných 30 behov pre prvú úroveň.

Tabuľka 14: Úspešnosť jednotlivých častí generovacej pipeline

Testovaná časť	Počet behov	Úspešné	Fallback	Úspešnosť
LLM plán	30	30	0	100,00%
Pozadie	30	29	0	96,67%
Terénna textúra	30	28	1	93,33%
Sprite hráča	30	29	0	96,67%
Sprity nepriateľov	90	87	0	96,67%
Zberateľný objekt	30	29	0	96,67%
Celá pipeline	30	29	0	96,67%

V riadku *Sprity nepriateľov* sú zahrnuté tri typy nepriateľov: pozemný, lietajúci a strieľajúci nepriateľ. Pri 30 behoch tak vzniklo spolu 90 testovaných nepriateľských sprite výstupov.

6.5 Časová náročnosť generovania

Keďže systém pracuje s lokálne spustenými modelmi, čas generovania predstavuje dôležitý faktor používateľskej skúsenosti. Meranie sa zameriavalo na trvanie jednotlivých krokov generovania aj na celkový čas prípravy jednej úrovne.

Tabuľka 15: Časová náročnosť jednotlivých krokov generovania

Krok	Min. čas [s]	Priemerný čas [s]	Max. čas [s]
LLM plán	3,90	8,40	24,72
Pozadie	5,44	7,22	11,82
Terénna textúra	3,21	4,16	11,71
Sprite hráča	3,86	4,34	8,63
Nepriatelia	11,87	14,09	27,43
Zberateľný objekt/projektíl	7,83	9,02	13,39
Celá úroveň	38,95	47,29	65,33

Riadok *Nepriatel* predstavuje súčet časov generovania pozemného, lietajúceho a strieľajúceho nepriateľa. Riadok *Zberateľný objekt/projektil* predstavuje súčet času generovania zberateľného objektu a projektilu. Výsledky ukazujú, že najväčšiu časť celkového času tvorí generovanie obrazových assetov, zatiaľ čo samotné vytvorenie plánu jazykovým modelom je kratšie, ale má vyšší rozptyl medzi jednotlivými behmi.

6.6 Kvalitatívne hodnotenie generovaných výstupov

Okrem technickej úspešnosti a časovej náročnosti bolo potrebné posúdiť aj vizuálnu použiteľnosť generovaných výstupov. Hodnotenie sa zameriavalo na to, či jednotlivé výstupy zodpovedali zadanej téme, či boli dostatočne čitateľné v 2D hernej scéne a či ich bolo možné použiť v Unity bez zásadných ručných úprav. Pri spritoch bola dôležitá najmä ich oddeliteľnosť od pozadia, čitateľná silueta a primeraná mierka vzhľadom na ostatné objekty v scéne. Pri pozadiach a terénnych textúrach sa sledovalo, či výstup nepôsobí ako samostatná ilustrácia, ale ako prvok vhodný na použitie v hernom prostredí. Súčasťou hodnotenia bola aj vizuálna konzistentnosť medzi jednotlivými výstupmi a výskyt nežiaducich prvkov, napríklad artefaktov, rámkov, textu v obrázku alebo nesprávnej perspektívy.

Hodnotenie bolo vykonané manuálne na základe exportovaných obrázkov vytvorených počas testovania. Cieľom nebolo určiť absolútnu kvalitu použitého obrazového modelu, ale identifikovať typické problémy, ktoré sa objavujú pri použití generovaných výstupov ako herných prvkov. Na spresnenie hodnotenia bola použitá bodová škála od 1 do 5, kde hodnota 1 označuje nepoužiteľný výstup a hodnota 5 označuje výstup, ktorý je tematicky vhodný, vizuálne čitateľný a použiteľný v Unity bez zásadných úprav.

Tabuľka 16: Bodové hodnotenie kvality generovaných výstupov

Typ výstupu	Tematická zhoda	Technická použiteľnosť	Čitateľnosť v scéne	Priemer
Pozadie	5	4	5	4,7
Terénna textúra	3	3	3	3,0
Sprite hráča	4	4	4	4,0
Sprity nepriateľov	4	4	3	3,7
Zberateľný objekt	4	3	3	3,3

Hodnoty v tabuľke predstavujú manuálne kvalitatívne hodnotenie exportovaných výstupov na škále od 1 do 5. Hodnotenie nepredstavuje používateľskú štúdiu, ale manuálne posúdenie výstupov autorom práce z hľadiska ich použiteľnosti v navrhovanom prototypu. Najvyššie hodnotenie dosiahli pozadia, ktoré vo väčšine prípadov dobre

zodpovedali zadanej téme a boli použiteľné ako vizuálny základ úrovne. Dobré výsledky dosahovali aj sprity hráča, ktoré mali vo väčšine behov čitateľnú siluetu a boli vhodne oddelené od pozadia.

Najslabšie hodnotenie dosiahli terénne textúry a menšie herné objekty. Pri terénnych textúrach model často nevytvoril čistý povrchový materiál, ale skôr detailný objekt, perspektívny záber alebo scénický obrázok. Pri zberateľných objektoch a projektiloch bol problém najmä v prílišnej detailnosti a nejednoznačnej funkcii objektu pri malom zobrazení v scéne. Sprity nepriateľov boli tematicky variabilné, ale ich štýl a mierka neboli vždy konzistentné medzi jednotlivými typmi nepriateľov.

Na ilustráciu rozdielov medzi jednotlivými typmi výstupov sú na obrázku 9 uvedené ukážky generovaných prvkov použitých pri hodnotení.



Obr. 9: Ukážky generovaných výstupov použitých pri kvalitatívnom hodnotení

Z ukážok je vidieť, že jednotlivé typy výstupov majú rozdielne požiadavky na použiteľnosť. Pozadie môže byť použiteľné aj vtedy, keď obsahuje viac detailov, pretože slúži ako vizuálny podklad úrovne. Naopak, sprite postavy alebo nepriateľa musí byť dobre oddelený od pozadia, mať čitateľný tvar a nesmie obsahovať výrazné rušivé prvky. Terénna textúra musí pôsobiť ako povrchový materiál, nie ako samostatná krajinná scéna. Preto bolo pri praktickom použití výstupov potrebné kombinovať generovanie s následným spracovaním a výberom vhodných výsledkov.

6.7 Ukážky výsledných scén

Na obrázkoch 10, 11 a 12 sú zobrazené ukážky výsledných scén vygenerovaných pre tri rozdielne témy. Cieľom ukážok je vizuálne demonštrovať, ako sa textové zadanie premietlo do výsledného vzhľadu úrovne.



Obr. 10: Výsledná scéna pre tému *Cave*.



Obr. 11: Výsledná scéna pre tému *Volcano*.



Obr. 12: Výsledná scéna pre tému *Forest*.

Z ukážok je vidieť, že systém dokáže zachovať základnú tematickú identitu zadania. Pri téme *Cave* dominuje jaskynné prostredie s chladnou farebnou paletou, pri téme *Volcano* sopečné prostredie s výraznými červenými a oranžovými tónmi a pri téme *Forest* lesné prostredie so zelenou farebnou paletou.

6.8 Diskusia výsledkov a limity

Výsledky testovania ukazujú, že prepojenie Unity s lokálne spustenými generatívnymi modelmi je pre účely prototypu realizovateľné. Väčšina častí procesu generovania dosiahla úspešnosť nad 90 %, pričom najstabilnejšou časťou bolo vytvorenie štruktúrovaného plánu pomocou jazykového modelu. Najnižšiu úspešnosť vykázala terénna textúra, pri ktorej sa objavili problémy s dostupnosťou alebo použiteľnosťou výstupu.

Kvalitatívne hodnotenie ukázalo, že najlepšie výsledky dosahovali pozadia a sprite hráča. Pozadia vo väčšine prípadov tematicky zodpovedali zadaniu a boli dobre použiteľné ako vizuálny základ úrovne. Sprite hráča bol po odstránení pozadia vo viacerých prípadoch použiteľný ako samostatný objekt v scéne. Slabšie výsledky dosahovali terénne textúry, pri ktorých model niekedy vytváral scénické alebo perspektívne obrázky namiesto povrchového materiálu. Pri nepriateľoch, zberateľných objektoch a projektiloch sa najčastejšie prejavovali problémy s mierkou, čitateľnosťou a konzistentnosťou vizuálneho štýlu.

Hlavným obmedzením riešenia je stochastický charakter generatívnych modelov. Rovnaké zadanie nemusí pri opakovanom spustení viesť k rovnako kvalitnému výsledku.

Ďalším obmedzením je časová náročnosť lokálneho generovania. Priemerný čas prípravy jednej úrovne bol 47,29 sekundy a najpomalší beh trval 65,33 sekundy. Pre prototyp je takýto čas prijateľný, no pri produkčnom použití by bolo potrebné riešiť ukladanie vygenerovaných výstupov do vyrovnávacej pamäte, predgenerovanie alebo zníženie počtu generovaných prvkov.

Záver

Cieľom bakalárskej práce bolo preskúmať možnosti využitia generatívnej umelej inteligencie pri tvorbe dynamického obsahu v prostredí Unity a implementovať prototyp, ktorý tento prístup prakticky demonštruje. V práci bol spracovaný prehľad generatívnych modelov podľa modality, analyzované boli spôsoby zadávania vstupov, spracovania výstupov, existujúce aplikácie využívajúce GenAI a možnosti integrácie modelov do Unity.

Výsledkom práce je prototyp 2D platformovej hry, v ktorom používateľ zadáva textovú tému úrovne. Jazykový model na jej základe vytvorí štruktúrovaný plán s promptmi a hernými parametrami. Obrazový model následne generuje vizuálne assety, ktoré sú spracované a aplikované do procedurálne vytvorenej Unity scény.

Implementácia ukázala, že generované výstupy nemožno používať priamo bez ďalšieho spracovania. Dôležitou súčasťou riešenia bola validácia, postprocessing sprite assetov, odstránenie pozadia a náhradné mechanizmy pri neúspešnom alebo nekvalitnom výstupe. Základnú hrateľnosť preto zabezpečuje procedurálny generátor a AI vrstva slúži najmä na rozšírenie tematickej a vizuálnej variability.

Testovanie potvrdilo, že prototyp dokáže vytvoriť hrateľnú úroveň pre rôzne tematické zadania. Zároveň sa ukázalo, že lokálne generovanie obrazových assetov je časovo náročné a kvalita výstupov závisí od promptov, použitého checkpointu, typu assetu a následného spracovania. Najproblematickejšie boli najmä sprite assety a terénne textúry, ktoré musia spĺňať technické požiadavky na čitateľnosť, mierku a použiteľnosť v scéne.

Ďalší vývoj by sa mohol zamerať na porovnanie lokálne spustených modelov so vzdialenými cloudovými službami. Vzdialené modely by mohli priniesť kvalitnejšie a konzistentnejšie výstupy, nižšie nároky na hardvér používateľa a jednoduchšie nasadenie. Nevýhodou by však bola závislosť od internetového pripojenia, dostupnosti cloudovej služby, externých API a prípadných prevádzkových nákladov.

Literatúra

1. UNITY TECHNOLOGIES. *Unity Engine* [online]. 2025. [cit. 2025-11-05]. Dostupné z: <https://unity.com/products/unity-engine>.
2. UNITY TECHNOLOGIES. *Glossary — Asset terms (Asset)* [online]. 2025. [cit. 2025-10-30]. Dostupné z: <https://docs.unity3d.com/6000.2/Documentation/Manual/Glossary.html#SectionAssets>.
3. UNITY TECHNOLOGIES. *Unity User Manual - Asset workflow* [online]. 2025. [cit. 2025-10-30]. Dostupné z: <https://docs.unity3d.com/6000.0/Documentation/Manual/AssetWorkflow.html>.
4. UNITY TECHNOLOGIES. *Explore the Unity Editor* [online]. 2025. [cit. 2026-06-03]. Dostupné z: <https://learn.unity.com/tutorial/explore-the-unity-editor-1?language=en>.
5. GOOGLE CLOUD. *What Is Artificial Intelligence?* [online]. 2025. [cit. 2025-11-05]. Dostupné z: <https://cloud.google.com/learn/what-is-artificial-intelligence>.
6. YANNAKAKIS, G. N.; TOGELIUS, J. *Artificial Intelligence and Games*. Springer Cham, 2018. ISBN 978-3-319-63519-4. Dostupné z DOI: 10.1007/978-3-319-63519-4.
7. YANNAKAKIS, G. N.; TOGELIUS, J. *Artificial Intelligence and Games 2*. Springer Cham, 2025. ISBN 978-3-031-83347-2. Dostupné z DOI: 10.1007/978-3-031-83347-2.
8. AMAZON WEB SERVICES. *What is Generative AI?* [online]. 2025. [cit. 2025-11-05]. Dostupné z: <https://aws.amazon.com/what-is/generative-ai/>.
9. ROMBACH, R.; BLATTMANN, A.; LORENZ, D.; ESSER, P.; OMMER, B. High-Resolution Image Synthesis with Latent Diffusion Models. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, s. 10684–10695.
10. ALTEXSOFT EDITORIAL TEAM. *AI Image Generators: DALL-E, Stable Diffusion, Adobe Firefly, and Midjourney Compared* [online]. 2024. [cit. 2026-06-03]. Dostupné z: <https://www.altexsoft.com/blog/top-ai-image-generators/>.
11. AMAZON WEB SERVICES. *What is an API?* [online]. 2024. [cit. 2025-11-13]. Dostupné z: <https://aws.amazon.com/what-is/api/>.

12. LAWANDE, S.; THAKUR, P.; JOSHI, O.; PAWAR, A. A Systematic Review and Analysis of Intelligence-Based Pathfinding Algorithms in the Field of Video Games. *Applied Sciences*. 2022. Dostupné z DOI: 10.3390/app12115499.
13. JULIANI, A. et al. Unity: A General Platform for Intelligent Agents. *arXiv preprint arXiv:1809.02627*. 2018. Dostupné tiež z: <https://arxiv.org/abs/1809.02627>.
14. UNITY TECHNOLOGIES. *Unity AI products* [online]. 2026. [cit. 2026-05-25]. Dostupné z: <https://docs.unity3d.com/Packages/com.unity.ai.assistant@2.11/manual/index.html>.
15. SCENARIO. *Scenario Documentation* [online]. 2026. [cit. 2026-05-25]. Dostupné z: <https://docs.scenario.com/>.
16. ROSEBUD AI. *Rosebud AI: Make 3D Games & Worlds with Vibe Coding* [online]. 2026. [cit. 2026-05-25]. Dostupné z: <https://lab.rosebud.ai/blog/rosebud-ai-interface-guide>.
17. AUTOMATIC1111. *Stable Diffusion WebUI Wiki* [online]. 2026. [cit. 2026-05-25]. Dostupné z: <https://github.com/AUTOMATIC1111/stable-diffusion-webui/wiki>.
18. OLLAMA. *Ollama Documentation* [<https://docs.ollama.com/>]. 2026. [cit. 2026-05-24].
19. LM STUDIO. *LM Studio Documentation* [online]. 2025. [cit. 2026-05-30]. Dostupné z: <https://lmstudio.ai/docs/app>.
20. LLAMA.CPP. *llama.cpp Documentation* [online]. 2025. [cit. 2026-05-30]. Dostupné z: <https://llama-cpp.com/>.
21. AUTOMATIC1111. *Stable Diffusion Web UI* [<https://github.com/AUTOMATIC1111/stable-diffusion-webui>]. 2026. [cit. 2026-05-24].
22. COMFYUI. *ComfyUI Documentation* [online]. 2025. [cit. 2026-05-30]. Dostupné z: <https://docs.comfy.org/>.
23. INVOKEAI. *InvokeAI Documentation* [online]. 2025. [cit. 2026-05-30]. Dostupné z: <https://invoke.ai/>.
24. OLLAMA. *mistral:7b* [<https://ollama.com/library/mistral:7b>]. 2026. [cit. 2026-05-24].

Použitie nástrojov umelej inteligencie

OpenAI (2026), ChatGPT 5.5, vyhľadávanie zdrojov a dopĺňujúcich informácií k téme práce.

OpenAI (2026), ChatGPT 5.5, stylistická úprava textu, terminologické zjednotenie a finálna revízia práce.

Anthropic (2026), Claude Sonnet 4.6, pomoc pri návrhu, písaní a úprave zdrojového kódu prototypu.

Dodatok A: Súborová štruktúra projektu

Príloha práce obsahuje bakalársku prácu vo formáte PDF a archív `Bakalarka_Unity.zip`, v ktorom sa nachádza Unity projekt implementovaného prototypu. Archív obsahuje hlavné adresáre potrebné na otvorenie projektu v prostredí Unity. Automaticky generované adresáre, napríklad `Library` alebo `Logs`, nie sú súčasťou odovzdaného archívu, pretože ich Unity dokáže pri otvorení projektu vytvoriť znovu.

`BP25_Hrušovský.pdf` bakalárska práca vo formáte PDF.

`Bakalarka_Unity.zip` komprimovaný priečinnok Unity projektu.

`/Assets` hlavný adresár projektu obsahujúci scény, skripty, nastavenia a ostatné assety použité v prototypu.

`/Assets/Scenes` scény Unity projektu. Nachádzajú sa tu najmä scény *MainMenu* a *Game*.

`/Assets/Scripts` zdrojové skripty implementovaného prototypu. Obsahujú časti pre komunikáciu s AI backendmi, generovanie úrovne, spracovanie výstupov, hernú logiku a používateľské rozhranie.

`/Packages` adresár obsahujúci zoznam použitých Unity balíkov a závislostí projektu.

`/ProjectSettings` adresár s nastaveniami Unity projektu.

Dodatok B: Používateľský manuál

Táto príloha opisuje základný postup sprevádzkovania Unity projektu. Projekt využíva lokálne spustené nástroje Ollama, Stable Diffusion WebUI AUTOMATIC1111 a rembg. Unity s nimi komunikuje cez lokálne HTTP rozhrania.

B.1 Potrebne nástroje

Na spustenie projektu sú potrebné tieto nástroje:

- Unity Hub: <https://unity.com/download>
- Unity Editor: <https://unity.com/releases/editor/archive>
- Git for Windows: <https://git-scm.com/install/windows>
- Python 3.10.6: <https://www.python.org/downloads/release/python-3106/>
- Ollama: <https://ollama.com/download>
- AUTOMATIC1111 Stable Diffusion WebUI: <https://github.com/AUTOMATIC1111/stable-diffusion-webui>
- DreamShaper checkpoint: <https://civitai.com/models/4384/dreamshaper>
- rembg: <https://github.com/danielgatis/rembg>

Projekt bol testovaný vo verzii Unity 6000.3.5f2.

B.2 Príprava Unity projektu

Projekt sa otvorí cez Unity Hub:

Projects -> Add -> Add project from disk

V nastaveniach *Build Settings* musia byť pridané scény *MainMenu* a *Game*. Ako prvá má byť nastavená scéna *MainMenu*.

B.3 Príprava Ollama

Ollama slúži na lokálne spustenie jazykového modelu. Po inštalácii je potrebné stiahnuť model:

```
ollama pull mistral:7b
```

Dostupnosť modelu možno overiť príkazom:

```
ollama list
```

Ollama beží štandardne na adrese:

```
http://localhost:11434
```

B.4 Príprava Stable Diffusion WebUI

Stable Diffusion WebUI AUTOMATIC1111 slúži na generovanie obrazových výstupov. Po nainštalovaní Git a Python 3.10.6 sa WebUI stiahne príkazom:

```
git clone https://github.com/AUTOMATIC1111/stable-diffusion-webui.git
```

V súbore `webui-user.bat` je potrebné nastaviť:

```
set COMMANDLINE_ARGS=--api
```

WebUI sa spustí súborom:

```
webui-user.bat
```

Po spustení musí byť dostupná adresa:

```
http://127.0.0.1:7860
```

API dokumentácia je dostupná na:

```
http://127.0.0.1:7860/docs
```

B.5 Pridanie checkpointu

Checkpoint `dreamshaper_8` je potrebné vložiť do priečinka:

```
stable-diffusion-webui/models/Stable-diffusion/
```

Po vložení checkpointu je potrebné WebUI reštartovať alebo obnoviť zoznam modelov.

B.6 Príprava rembg

Nástroj `rembg` sa používa na odstránenie pozadia zo spritov. Odporúčané je vytvoriť samostatné Python prostredie:

```
python -m venv venv
.\venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install "rembg[cpu,cli]" pillow
```

Server sa spustí príkazom:

```
rembg s --host 127.0.0.1 --port 7000
```

Unity používa adresu:

```
http://127.0.0.1:7000/api/remove
```

Dostupnosť servera možno overiť na:

```
http://127.0.0.1:7000/api
```

B.7 Poradie spúšťania

Pred spustením hry je potrebné spustiť:

1. Ollama,
2. Stable Diffusion WebUI s parametrom `-api`,
3. `rembg` server,
4. Unity projekt.

Použité lokálne adresy:

- OllamaClient: `http://localhost:11434`
- StableDiffusion: `http://127.0.0.1:7860`
- RembgClient: `http://127.0.0.1:7000`

B.8 Spustenie hry

V Unity sa otvorí scéna *MainMenu*. Používateľ zadá tému, zvolí obtiažnosť a spustí hru. Unity následne vytvorí plán pomocou Ollama, vygeneruje obrazové výstupy cez Stable Diffusion WebUI, odstráni pozadie pomocou `rembg` a aplikuje výsledné prvky do procedurálne vytvorenej úrovne.

B.9 Najčastejšie problémy

Ak sa negenerujú obrazové výstupy, treba overiť, či Stable Diffusion WebUI beží s parametrom `-api`:

```
http://127.0.0.1:7860/docs
```

Ak sa nevytvorí plán úrovne, treba overiť, či beží Ollama a či je dostupný model:

```
ollama list
```

Ak `rembg` hlási chybu `No module named onnxruntime`, je potrebné nainštalovať:

```
python -m pip install onnxruntime
```

Ak sa checkpoint nezobrazuje vo WebUI, treba skontrolovať jeho umiestnenie v priečinku:

```
stable-diffusion-webui/models/Stable-diffusion/
```